



Henrique Campanha Garcia

Desenvolvimento de um Veículo Robótico Autônomo de Pequeno Porte Baseado em ESP32

São José dos Campos, SP

Henrique Campanha Garcia

Desenvolvimento de um Veículo Robótico Autônomo de Pequeno Porte Baseado em ESP32

Trabalho de conclusão de curso apresentado ao Instituto de Ciência e Tecnologia – UNIFESP, como parte das atividades para obtenção do título de Bacharel em Ciência da Computação.

Universidade Federal de São Paulo – UNIFESP

Instituto de Ciência de Tecnologia

Bacharelado em Ciência da Computação

Orientador: Prof. Dr. André Marcorin de Oliveira

São José dos Campos, SP

Junho de 2026

Henrique Campanha Garcia

Desenvolvimento de um Veículo Robótico Autônomo de Pequeno Porte Baseado em ESP32

Trabalho de conclusão de curso apresentado ao Instituto de Ciência e Tecnologia – UNIFESP, como parte das atividades para obtenção do título de Bacharel em Ciência da Computação.

Prof. Dr. André Marcorin de Oliveira
Orientador

Prof. Dr. Sérgio Ronaldo Barros dos Santos
Convidado 1

Professor
Convidado 2

Professor
Convidado 3

São José dos Campos, SP
Junho de 2026

Dedico este trabalho ao meu orientador, cuja orientação foi essencial para o desenvolvimento deste projeto.

Seus esclarecimentos, especialmente na compreensão e aplicação dos algoritmos envolvidos, foram fundamentais para que eu pudesse superar desafios técnicos e aprofundar meus conhecimentos em áreas que antes me pareciam complexas.

Agradeço pela paciência, disponibilidade e incentivo contínuo ao longo deste processo.

Dedico este trabalho também a todos que, de alguma forma contribuíram para a realização deste projeto.

Muito obrigado.

Agradecimentos

A realização deste trabalho foi possível graças ao apoio e à colaboração de diversas pessoas e instituições, às quais expresso minha sincera gratidão.

Em primeiro lugar, agradeço ao meu orientador, Prof. Dr. André Marcorin de Oliveira, pelo constante apoio, paciência e pelas valiosas orientações técnicas, especialmente no que diz respeito à compreensão e à implementação dos algoritmos utilizados neste projeto. Sua experiência, dedicação e disponibilidade foram fundamentais para o desenvolvimento desta pesquisa.

Estendo meus agradecimentos à Universidade Federal de São Paulo (UNIFESP), instituição de ensino reconhecida pela excelência acadêmica, pelo compromisso com a formação de profissionais éticos, críticos e preparados para os desafios da sociedade contemporânea.

Aos colegas e amigos que me acompanharam ao longo da graduação, em especial Thiago Capuano e Vinicius Matumoto, sou grato pelos diálogos construtivos, pela colaboração espontânea e, sobretudo, pela amizade que tornou essa caminhada mais leve e motivadora.

À minha família, em especial aos meus pais, Margarete e Renato Garcia, agradeço pelo amor incondicional, incentivo constante e por acreditarem em mim mesmo diante das dificuldades. Sem o apoio de vocês, este trabalho certamente não teria se concretizado.

Por fim, agradeço a todos aqueles que, direta ou indiretamente, contribuíram para a realização deste projeto.

*“Não vos amoldeis às estruturas deste mundo,
mas transformai-vos pela renovação da mente,
a fim de distinguir qual é a vontade de Deus:
o que é bom, o que Lhe é agradável, o que é perfeito.
(Bíblia Sagrada, Romanos 12, 2)*

Resumo

Este trabalho apresenta o projeto e desenvolvimento de um veículo robótico autônomo de pequeno porte, concebido para realizar tarefas de transporte de objetos em ambientes internos. O sistema é fundamentado no microcontrolador ESP32, que centraliza o processamento e a conectividade. A percepção do ambiente e a locomoção precisa são viabilizadas pela integração de múltiplos sensores e atuadores, incluindo um magnetômetro QMC5883L para referência de orientação, um sensor de distância *Time-of-Flight* (ToF) VL53L0X para detecção frontal de obstáculos e encoders ópticos para odometria. O controle de movimento dos motores DC é realizado por meio de uma ponte H com modulação por largura de pulso (PWM), e a estabilidade da trajetória é garantida por um algoritmo de controle Proporcional, Integral e Derivativo (PID). A autonomia de navegação é alcançada por uma estratégia híbrida que combina planejamento de rota global, utilizando o algoritmo A* sobre um mapa de grid, com um controle reativo local para desvio de obstáculos em tempo real. A interação com o usuário é realizada através de uma interface web (WebServer) embarcada, que permite a designação de destinos e o monitoramento do robô via Wi-Fi. O projeto visa entregar um protótipo funcional, de baixo custo e com arquitetura modular, servindo como uma plataforma educacional e de pesquisa em robótica móvel, sistemas embarcados e Internet das Coisas (IoT).

Palavras-chave: Robô Autônomo. ESP32. Controle PID. Navegação Robótica. Mapeamento em Grid. Sistemas Embarcados.

Abstract

This work presents the design and development of a small-scale autonomous robotic vehicle, designed to perform object transportation tasks in indoor environments. The system is based on the ESP32 microcontroller, which centralizes processing and connectivity. Environmental perception and precise locomotion are enabled by the integration of multiple sensors and actuators, including a QMC5883L magnetometer for heading reference, a VL53L0X Time-of-Flight (ToF) distance sensor for frontal obstacle detection, and optical encoders for odometry. The motion control of the DC motors is performed using an H-bridge with Pulse Width Modulation (PWM), and trajectory stability is ensured by a Proportional, Integral, and Derivative (PID) control algorithm. Navigation autonomy is achieved through a hybrid strategy that combines global path planning, using the A* algorithm on a grid map, with local reactive control for real-time obstacle avoidance. User interaction is carried out through an embedded web interface (WebServer), which allows for destination designation and robot monitoring via Wi-Fi. The project aims to deliver a functional, low-cost prototype with a modular architecture, serving as an educational and research platform in mobile robotics, embedded systems, and the Internet of Things (IoT).

Keywords: Autonomous Robot. ESP32. PID Control. Robotic Navigation. Grid Mapping. Embedded Systems.

Lista de ilustrações

Figura 1 – Diagrama de pinagem do ESP32.	26
Figura 2 – Funcionamento da ponte H para diferentes sentidos de rotação.	29
Figura 3 – Módulo magnetômetro QMC5883L.	30
Figura 4 – Sensor VL53L0X	31
Figura 5 – Exemplo simplificado de mapeamento em grid.	34
Figura 6 – Mapa 7x7 utilizado para validação da navegação, com células de 30 cm × 30 cm, células ocupadas rubricadas e representação da pose inicial do robô em (1, 1).	35
Figura 7 – Exemplo de trajetória do robô durante o processo de mapeamento.	36
Figura 8 – Arquitetura geral do sistema robótico embarcado.	40
Figura 9 – Organização revisada dos componentes utilizados no protótipo do robô móvel.	42
Figura 10 – Diagrama simplificado da malha de controle de velocidade dos motores.	48
Figura 11 – Diagrama simplificado da malha de controle de direção do robô.	49
Figura 12 – Exemplo de caminho traçado pelo algoritmo A* sobre um mapa de ocupação.	52
Figura 13 – Mapa 7x7 utilizado nos testes de navegação por células.	62
Figura 14 – Interface principal de controle do robô durante os testes.	66
Figura 15 – Interface de monitoramento e logs em tempo real.	67

Lista de abreviaturas e siglas

ADC	Conversor Analógico-Digital (Analog-to-Digital Converter)
AUV	Veículo Submerso Autônomo (Autonomous Underwater Vehicle)
DC	Corrente Contínua (Direct Current)
GPIO	Pinos de Entrada/Saída de Propósito Geral (General-Purpose Input/Output)
HTTP	Protocolo de Transferência de Hipertexto (Hypertext Transfer Protocol)
IMU	Unidade de Medição Inercial (Inertial Measurement Unit)
IoT	Internet das Coisas (Internet of Things)
IHM	Interface Homem-Máquina
I2C	Inter-Integrated Circuit (padrão de comunicação)
PID	Controlador Proporcional-Integral-Derivativo
PRM	Mapa de Rotas Probabilístico (Probabilistic Roadmap)
PWM	Modulação por Largura de Pulso (Pulse Width Modulation)
RAS	Sistemas Robóticos e Autônomos (Robotic and Autonomous Systems)
RMA	Robôs Móveis Autônomos
RTOS	Sistema Operacional de Tempo Real (Real-Time Operating System)
SLAM	Localização e Mapeamento Simultâneos (Simultaneous Localization and Mapping)
SPI	Interface Periférica Serial (Serial Peripheral Interface)
ToF	Tempo de Voo (Time-of-Flight)
UART	Transmissor/Receptor Assíncrono Universal (Universal Asynchronous Receiver-Transmitter)
VL53L0X	Sensor ToF da STMicroelectronics
Wi-Fi	Wireless Fidelity (Tecnologia de rede sem fio)
WebServer	Servidor Web embarcado

ESP32 Microcontrolador com Wi-Fi e Bluetooth integrados da Espressif Systems

QMC5883L Magnetômetro digital de três eixos utilizado como referência de orientação magnética

A* Algoritmo A-star para planejamento de caminhos

Sumário

1	INTRODUÇÃO	19
1.1	Objetivos	20
1.1.1	Objetivo Geral	20
1.1.2	Objetivos Específicos	20
1.2	Organização do Texto	21
2	REVISÃO BIBLIOGRÁFICA	23
3	FUNDAMENTAÇÃO TEÓRICA	25
3.1	Sistemas Embarcados e IoT	25
3.2	FreeRTOS em Sistemas Embarcados	27
3.3	Controle de Motores DC e Ponte H	28
3.4	Orientação Magnética com QMC5883L	29
3.5	Sensores Time-of-Flight: VL53L0X	31
3.6	Algoritmos PID e Odometria	32
3.7	Mapeamento em Grid e Planejamento de Rotas	33
4	MATERIAIS E MÉTODOS	39
4.1	Arquitetura do Sistema e Prototipagem de Hardware	39
4.1.1	Prototipagem e Ajustes de Hardware	41
4.2	Ambiente de Desenvolvimento	42
4.2.1	Execução concorrente com FreeRTOS	43
4.2.2	Bibliotecas Utilizadas	43
4.2.3	Bibliotecas Customizadas	44
4.3	Estratégias de Controle e Navegação	45
4.3.1	Odometria e Estimativa de Posição	45
4.3.2	Controle de Movimento via PID	48
4.3.3	Sensoriamento e Mapeamento do Ambiente	51
4.3.4	Planejamento de Rota com Algoritmo A*	52
4.3.5	Execução da Navegação	53
4.3.6	Calibração de Giro e Rejeição de Leituras Instáveis	54
4.4	Interface Web e Sistema de Arquivos Flash	55
4.5	Integração e Montagem Mecânica do Robô	55
4.6	Descrição dos Testes	55
5	RESULTADOS	61

5.1	Mapa de Teste e Planejamento de Rotas	61
5.2	Resultados do Deslocamento Linear	63
5.3	Resultados do Controle de Rotação	64
5.4	Interface Web e Monitoramento	66
5.5	Síntese dos Resultados Obtidos	67
6	CONCLUSÃO	71
6.1	Próximos Passos	73
	REFERÊNCIAS	75

1 Introdução

A crescente evolução dos sistemas embarcados e da Internet das Coisas (IoT) tem impulsionado o desenvolvimento de soluções autônomas para aplicações cotidianas. Nesse cenário, veículos autônomos têm despertado interesse pela sua aplicabilidade em tarefas de transporte interno, contribuindo para a automação e otimização de processos, [Schilling et al. \(1997\)](#).

O campo da robótica móvel tem-se desenvolvido significativamente nas últimas décadas, especialmente com a popularização de plataformas modulares e reconfiguráveis [Moubarak e Ben-Tzvi \(2012\)](#) e robôs com capacidade de adaptação a ambientes complexos e dinâmicos [Schilling et al. \(1997\)](#). Nesse contexto, dentro da robótica, os agentes podem ser classificados de acordo com seu grau de autonomia, como por exemplo, robôs teleoperados, semi-autônomos e autônomos. Em todos esses tipos de robôs, e em especial para robôs com graus elevados de autonomia, o uso de sensores embarcados, como unidades de medição inerciais (IMUs) e sensores de distância, em conjunto com algoritmos de controle robustos, como o PID (Proporcional, Integral, Derivativa), têm sido essenciais para garantir precisão e estabilidade [Parween et al. \(2020\)](#).

Dentre as aplicações de sistemas robóticos autônomos, podem ser destacadas os robôs de resgate e salvamento [Calisi et al. \(2007\)](#), [Samir et al. \(2024\)](#), manutenção de submarina [Lemos \(2021\)](#), [Nauert e Kampmann \(2023\)](#), interface com seres humanos em museus [Cantucci, Marini e Falcone \(2024\)](#), [Silva e Souza \(2024\)](#), aplicações militares [Studies \(2021\)](#), [Automate.org \(2023\)](#), entre outros. Em especial, os trabalhos de [Nehmzow \(2012\)](#) e [Schilling et al. \(1997\)](#) destacam a importância de projetos didáticos e aplicáveis de veículos autônomos tanto em ambientes industriais quanto acadêmicos, reforçando a relevância de soluções acessíveis e flexíveis.

Dessa forma, este projeto visa desenvolver um veículo autônomo de pequeno porte com quatro rodas, baseado em um microcontrolador de baixo custo, equipado com magnetômetro, sensor de distância e encoders ópticos. O sistema será capaz de realizar deslocamentos autônomos utilizando mapeamento em grid e planejamento de rotas, com controle remoto por interface web.

A principal motivação do projeto é criar uma solução acessível e didática para transporte interno automatizado em ambientes acadêmicos, facilitando processos de gestão dentro da universidade. Além disso, o desenvolvimento do projeto vai proporcionar o aprendizado multidisciplinar. A contribuição esperada é o desenvolvimento de uma plataforma de baixo custo com potencial de aplicação em ambientes reais, além de servir como base para futuras pesquisas em robótica móvel e IoT.

1.1 Objetivos

Este trabalho tem como finalidade o desenvolvimento de um assistente robótico autônomo voltado à automação de tarefas de transporte de objetos em ambientes internos, como escritórios e residências. A proposta consiste em permitir que o usuário delegue, por meio de uma interface de controle, a entrega de itens a um destino previamente definido, sem a necessidade de deslocamento físico até o local de destino.

Além de propor uma solução prática para aplicações logísticas em pequena escala, o projeto busca consolidar e aplicar os conhecimentos teóricos adquiridos ao longo da graduação, com ênfase nas disciplinas de Sistemas Embarcados, Inteligência Artificial e Internet das Coisas (IoT). A iniciativa visa integrar esses conceitos por meio da construção de um sistema funcional, de baixo custo e alto valor educacional e tecnológico.

1.1.1 Objetivo Geral

Desenvolver um protótipo funcional de um veículo robótico autônomo de pequeno porte, dotado de quatro rodas, controlado por um microcontrolador ESP32, e equipado com magnetômetro QMC5883L, sensor de distância por tempo de voo (VL53L0X) e encoders ópticos para monitoramento da rotação das rodas. O sistema deverá ser capaz de se locomover autonomamente em ambientes internos, utilizando mapeamento baseado em grid, planejamento de rotas com A* e definição do destino via interface web.

1.1.2 Objetivos Específicos

- Projetar e montar a estrutura física do robô, incluindo os motores, a carcaça e a eletrônica embarcada;
- Implementar controle de velocidade e trajetória utilizando algoritmos PID com feedback dos encoders;
- Integrar o magnetômetro, o sensor de distância e os encoders ópticos ao sistema de controle e navegação;
- Desenvolver o módulo de mapeamento em grid e aplicar algoritmos de busca para planejamento de trajetórias (como o algoritmo A*);
- Implementar uma interface web acessível via rede Wi-Fi, permitindo a seleção de destinos pelo usuário;
- Realizar testes experimentais e validar o desempenho do sistema em ambiente controlado, analisando a precisão, a robustez e a viabilidade da proposta.

1.2 Organização do Texto

Este trabalho está estruturado da seguinte forma:

- **Capítulo 2 — Revisão Bibliográfica:** apresenta uma análise de trabalhos relacionados à robótica móvel autônoma, abordando diferentes aplicações e tecnologias que fundamentam o projeto.
- **Capítulo 3 — Fundamentação Teórica:** detalha os conceitos teóricos essenciais, incluindo sistemas embarcados, controle de motores, magnetômetro, sensores de distância, odometria por encoders, algoritmos PID, odometria e técnicas de mapeamento e planejamento de rotas.
- **Capítulo 4 — Materiais e Métodos:** descreve os componentes de hardware e as ferramentas de software utilizadas, a arquitetura do sistema, as estratégias de controle e navegação, e a metodologia de desenvolvimento do protótipo.
- **Capítulo 5 — Resultados:** apresenta a metodologia de teste, os registros utilizados, os resultados de deslocamento, rotação, detecção de obstáculos, interface web e as limitações observadas experimentalmente.
- **Capítulo 6 — Conclusão:** consolida as conclusões do trabalho, discute a viabilidade do protótipo e apresenta recomendações para trabalhos futuros.

2 Revisão Bibliográfica

Este capítulo apresenta uma análise de trabalhos relacionados ao desenvolvimento de robôs móveis autônomos, com foco na aplicação em ambientes internos e no transporte de pequenos objetos. São considerados aspectos como autonomia de navegação, controle embarcado, sensoramento, planejamento de rotas e usabilidade em contextos educacionais ou institucionais. Embora a área de robótica móvel autônoma seja ampla, buscou-se destacar estudos que, direta ou indiretamente, dialogam com os objetivos e desafios deste projeto.

Robôs aplicados a missões de busca e salvamento, como os descritos por [Calisi et al. \(2007\)](#), utilizam estratégias avançadas de exploração e navegação em ambientes não estruturados. Esses robôs são capazes de se orientar em locais desconhecidos e dinâmicos, utilizando sensores embarcados e algoritmos de mapeamento para garantir a segurança da operação. De forma complementar, [Samir et al. \(2024\)](#) apresentam uma revisão sistemática sobre robôs de resgate equipados com inteligência artificial, destacando a integração entre sensoramento preciso, aprendizado de máquina e planejamento adaptativo em cenários de alto risco. Apesar de operarem em contextos significativamente mais complexos, esses estudos fornecem subsídios importantes sobre navegação segura e sensoramento embarcado.

Ainda que este projeto se concentre em ambientes internos e controlados, como salas de aula e laboratórios, há paralelos metodológicos com outras aplicações robóticas. Trabalhos como os de [Lemos \(2021\)](#) e [Nauert e Kampmann \(2023\)](#) discutem o uso de veículos submersos autônomos (AUVs) para inspeção de estruturas submarinas, enfrentando desafios técnicos severos, como navegação em seis graus de liberdade e comunicação limitada. Apesar das diferenças no ambiente operacional, ambos os casos compartilham a necessidade de controle embarcado confiável, fusão sensorial e autonomia de locomoção — elementos também centrais para o presente projeto.

Em ambientes públicos e educacionais, [Cantucci, Marini e Falcone \(2024\)](#) investigaram o impacto da autonomia adaptativa de robôs na experiência dos visitantes em museus. Já [Silva e Souza \(2024\)](#) analisam o uso de robôs inteligentes como mediadores culturais em espaços expositivos. Esses estudos destacam a importância da interação com o usuário, da acessibilidade e da robustez na arquitetura de controle. O robô desenvolvido neste projeto, embora mais simples em termos de funcionalidade e escopo, compartilha o objetivo de ser uma ferramenta acessível, intuitiva e funcional em contextos institucionais e educacionais, promovendo uma experiência de uso descomplicada por meio de uma interface web.

Robôs autônomos também são amplamente empregados em aplicações militares e de defesa, conforme discutido em [Studies \(2021\)](#) e [Automate.org \(2023\)](#). Nesses cenários, robustez estrutural, inteligência situacional e operação remota segura são características críticas. Embora o

projeto aqui apresentado não possui finalidade bélica, ele adota conceitos estruturais semelhantes, como a navegação baseada em sensores, o controle de trajetória via PID e a operação remota via rede sem fio, adaptados para fins acadêmicos e com foco em baixo custo e reprodutibilidade.

No campo da robótica modular, [Moubarak e Ben-Tzvi \(2012\)](#) destacam a importância de plataformas reconfiguráveis, que possibilitam adaptações estruturais para diferentes tipos de missão. Embora o robô desenvolvido neste projeto não possua capacidade de reconfiguração física, adota uma arquitetura modular do ponto de vista do software e da organização dos componentes embarcados. O uso de bibliotecas customizadas, divididas por responsabilidade funcional, favorece a manutenção do sistema e sua expansão futura.

Em síntese, os trabalhos analisados contribuem para o embasamento técnico do projeto, oferecendo diferentes perspectivas sobre sensoriamento, controle e navegação em robôs móveis. O diferencial da proposta apresentada neste TCC reside na sua aplicação prática: a criação de um robô autônomo de pequeno porte, controlado por um ESP32, capaz de realizar o transporte de objetos em ambientes internos por meio de navegação baseada em mapeamento em grid, sensores de distância (VL53L0X), referência magnética de orientação (QMC5883L), algoritmo A* e interface web embarcada. Trata-se de uma solução voltada à didática, ao baixo custo e à possibilidade de replicação em contextos educacionais e institucionais.

3 Fundamentação Teórica

Este capítulo apresenta os fundamentos teóricos que sustentam o desenvolvimento do projeto, abrangendo os conceitos, tecnologias e métodos necessários para a construção de um veículo robótico autônomo com capacidade de navegação em ambiente interno. A estrutura modular do sistema envolve desde a integração de sensores e atuadores até o controle de movimento e planejamento de rotas.

Inicialmente, são abordados os conceitos de sistemas embarcados e Internet das Coisas (IoT), destacando-se suas arquiteturas e aplicações práticas, bem como a justificativa da escolha do microcontrolador ESP32, amplamente utilizado em projetos que exigem conectividade e capacidade computacional embarcada (Seção 3.1).

Na sequência, são discutidos os princípios de controle de motores de corrente contínua (DC) utilizando circuitos do tipo ponte H, responsáveis por permitir o acionamento bidirecional e o ajuste de velocidade dos motores (Seção 3.3). A Seção 3.6 trata da aplicação do controle PID para estabilização e controle de movimento, bem como do uso de odometria com encoders para estimativa de deslocamento do robô.

Também são explorados os recursos de orientação magnética (Seção 3.4), com destaque para o sensor QMC5883L, que fornece medições do campo magnético nos eixos X, Y e Z para estimativa de direção no plano horizontal. Em seguida, na Seção 3.5, discorre-se sobre sensores de distância baseados no princípio de *Time-of-Flight* (ToF), como o VL53L0X, cuja precisão e velocidade tornam-no adequado para detecção de obstáculos em tempo real.

Por fim, a Seção 3.7 apresenta as técnicas de representação do ambiente por meio de mapas em grid e os algoritmos de planejamento de rotas, como A* e Dijkstra, que permitem ao robô traçar trajetórias seguras e eficientes até o destino escolhido.

Esses tópicos compõem o corpo teórico necessário para a implementação do sistema proposto, oferecendo suporte conceitual e justificativa técnica para as decisões de projeto. A fundamentação apresentada a seguir baseia-se em referências consolidadas na literatura acadêmica e técnica, como Kurose e Ross (2009), entre outras.

3.1 Sistemas Embarcados e IoT

Sistemas embarcados consistem em plataformas computacionais especializadas, compostas por microcontroladores, firmware e, em alguns casos, sistemas operacionais de tempo real (RTOS), cujo propósito é executar tarefas específicas sob restrições rígidas de desempenho, consumo energético e confiabilidade. Essas plataformas integram, de forma sinérgica, componentes de hardware

e software, promovendo soluções otimizadas para diferentes finalidades. Por serem projetados para operar de maneira contínua, eficiente e, frequentemente, em tempo real, os sistemas embarcados desempenham papel central em diversos setores da tecnologia, como automação industrial, controle de processos, equipamentos médicos, monitoramento ambiental e dispositivos de Internet das Coisas (IoT), [Carvalho e Neto \(2024\)](#).

No contexto da IoT, a integração de sistemas embarcados com redes de comunicação sem fio, como Wi-Fi, Bluetooth e LoRa, potencializa a conectividade e a interoperabilidade entre dispositivos distribuídos. Essa conectividade torna possível a criação de ambientes inteligentes, capazes de reagir dinamicamente a estímulos do ambiente, compartilhar dados em tempo real e executar comandos remotos. Entretanto, tais sistemas exigem soluções que conciliem eficiência energética, segurança de dados e escalabilidade, aspectos críticos para aplicações em larga escala, como cidades inteligentes, agricultura de precisão e automação residencial.

Para o desenvolvimento deste projeto, foi selecionado o microcontrolador ESP32, do kit de desenvolvimento DOIT ESP32 Devkit V1, devido à sua ampla versatilidade, baixo custo e rica gama de funcionalidades embarcadas. O ESP32 possui conectividade Wi-Fi e Bluetooth integradas, múltiplos núcleos de processamento, suporte a múltiplos pinos GPIO, conversores analógico-digitais (ADCs), interfaces I2C, SPI e UART, além de timers e recursos de controle PWM, características que o tornam ideal para aplicações embarcadas modernas. A *Figura 1* apresenta o diagrama de pinagem da placa, destacando a disposição dos pinos e suas respectivas funcionalidades, informação essencial para a correta conexão dos periféricos utilizados no protótipo.

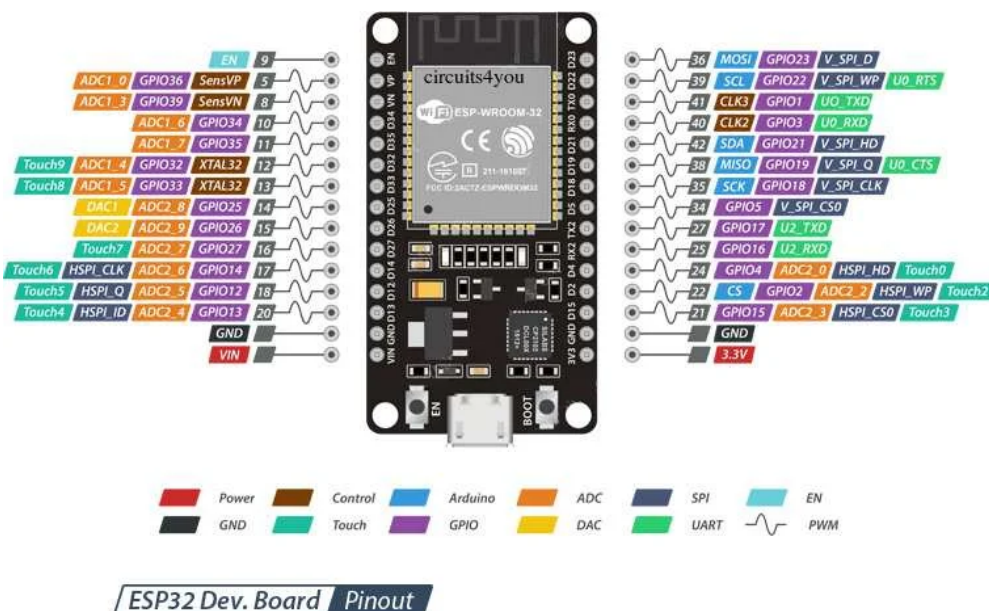


Figura 1 – Diagrama de pinagem do ESP32.

Fonte: lastminuteengineers.com

A escolha do ESP32 justifica-se, portanto, por sua capacidade de processamento superior, ampla conectividade, suporte a múltiplos protocolos de comunicação e integração de periféricos

essenciais, reduzindo a necessidade de componentes externos. Além disso, sua popularidade na comunidade de desenvolvimento embarcado proporciona vasta documentação e bibliotecas de código, facilitando o processo de implementação e depuração do sistema.

3.2 FreeRTOS em Sistemas Embarcados

Em sistemas embarcados, especialmente aqueles que precisam lidar simultaneamente com sensoriamento, controle de atuadores, comunicação e interface com o usuário, a execução sequencial de todas as tarefas pode se tornar limitada. Nesses casos, o uso de um sistema operacional de tempo real, ou RTOS (*Real-Time Operating System*), permite organizar o software em tarefas independentes, com prioridades, intervalos de execução e mecanismos de sincronização próprios.

O FreeRTOS é um sistema operacional de tempo real amplamente utilizado em microcontroladores por apresentar baixo consumo de memória, portabilidade e suporte a múltiplas arquiteturas embarcadas. Diferentemente de sistemas operacionais convencionais, seu objetivo não é fornecer uma interface completa de uso geral, mas sim permitir o escalonamento previsível de tarefas em sistemas com recursos computacionais limitados. Dessa forma, o FreeRTOS possibilita dividir a aplicação em diferentes fluxos de execução, como leitura de sensores, controle de motores, comunicação sem fio e atualização de estados internos do sistema.

No contexto do ESP32, o FreeRTOS é particularmente relevante, pois a própria plataforma de desenvolvimento utilizada pelo microcontrolador é baseada nesse sistema operacional. O ESP32 possui arquitetura com dois núcleos de processamento, permitindo que diferentes tarefas sejam distribuídas entre eles. Essa característica favorece a separação entre atividades críticas de controle, como leitura de encoders e atualização de comandos de motores, e atividades de comunicação, como atendimento à interface web e envio de telemetria.

Entre os principais recursos fornecidos pelo FreeRTOS estão a criação de tarefas, o escalonamento por prioridade, os atrasos temporizados, as filas de comunicação, os semáforos e os mecanismos de sincronização entre tarefas. Esses recursos permitem estruturar o software de forma modular, evitando que uma rotina bloqueante impeça a execução de outras partes importantes do sistema. Por exemplo, em um robô móvel, a leitura periódica de sensores deve ocorrer mesmo enquanto o sistema processa comandos recebidos pela interface web ou atualiza informações de monitoramento.

Neste trabalho, o uso do FreeRTOS está associado à organização das rotinas embarcadas do robô em tarefas de execução concorrente. Essa abordagem permite separar funções como aquisição de dados sensoriais, atualização da telemetria, controle de movimentação e comunicação com a interface web. Com isso, o sistema passa a responder de forma mais adequada a eventos simultâneos, como a detecção de obstáculos, a recepção de comandos externos e a necessidade de atualização contínua da posição estimada do robô.

Apesar dessas vantagens, o uso de tarefas concorrentes também exige cuidados adicionais. A troca de informações entre diferentes tarefas deve ser realizada de forma controlada, evitando condições de corrida, leituras inconsistentes ou disputas por recursos compartilhados. Por esse motivo, variáveis relacionadas ao estado do robô, leituras de sensores e comandos de movimentação devem ser tratadas com atenção, especialmente quando acessadas por múltiplas rotinas.

Assim, o FreeRTOS contribui para a organização do software embarcado ao permitir uma arquitetura baseada em tarefas, mais adequada à natureza concorrente do sistema robótico desenvolvido. Sua utilização favorece a separação entre sensoriamento, controle, navegação e comunicação, permitindo maior robustez e responsividade durante a execução dos testes experimentais.

3.3 Controle de Motores DC e Ponte H

Motores de corrente contínua (DC) são componentes amplamente utilizados em plataformas robóticas móveis, especialmente em projetos com sistemas embarcados. Isso se deve à sua construção relativamente simples, ao custo acessível e à capacidade de resposta rápida a comandos de controle. No entanto, para que seja possível controlar de forma eficaz tanto a direção quanto a velocidade desses motores, é necessário empregar circuitos de acionamento apropriados, sendo a ponte H uma das soluções mais utilizadas para esse fim.

A ponte H consiste em um arranjo eletrônico formado geralmente por quatro dispositivos semicondutores de chaveamento (transistores bipolares) dispostos de modo a formar uma topologia semelhante à letra “H”. Esse circuito permite o controle da polaridade da tensão aplicada aos terminais do motor, viabilizando, assim, o acionamento em ambos os sentidos de rotação. Tal característica é fundamental para o deslocamento de robôs autônomos, que requerem comandos precisos de avanço, recuo e curvas, os quais são obtidos por meio do controle individual de cada motor responsável pela movimentação.

Para controlar a velocidade de rotação, é frequentemente empregada a técnica de modulação por largura de pulso (PWM), que ajusta a potência média entregue ao motor pela variação do ciclo de trabalho de um sinal digital. Essa estratégia possibilita uma regulação contínua da velocidade, mantendo a eficiência energética do sistema e reduzindo o desgaste dos componentes eletromecânicos.

A ponte H, portanto, representa uma alternativa consolidada tanto em ambientes educacionais quanto em sistemas de aplicação prática, devido à sua confiabilidade e simplicidade de implementação. Sua interface com microcontroladores, como o ESP32, ocorre por meio de sinais digitais que controlam o estado de cada chave do circuito, permitindo a integração direta com algoritmos de controle embarcados.

A *Figura 2* ilustra os dois modos de operação da ponte H, demonstrando como a combinação das chaves eletrônicas determina a direção da corrente elétrica no motor, resultando em

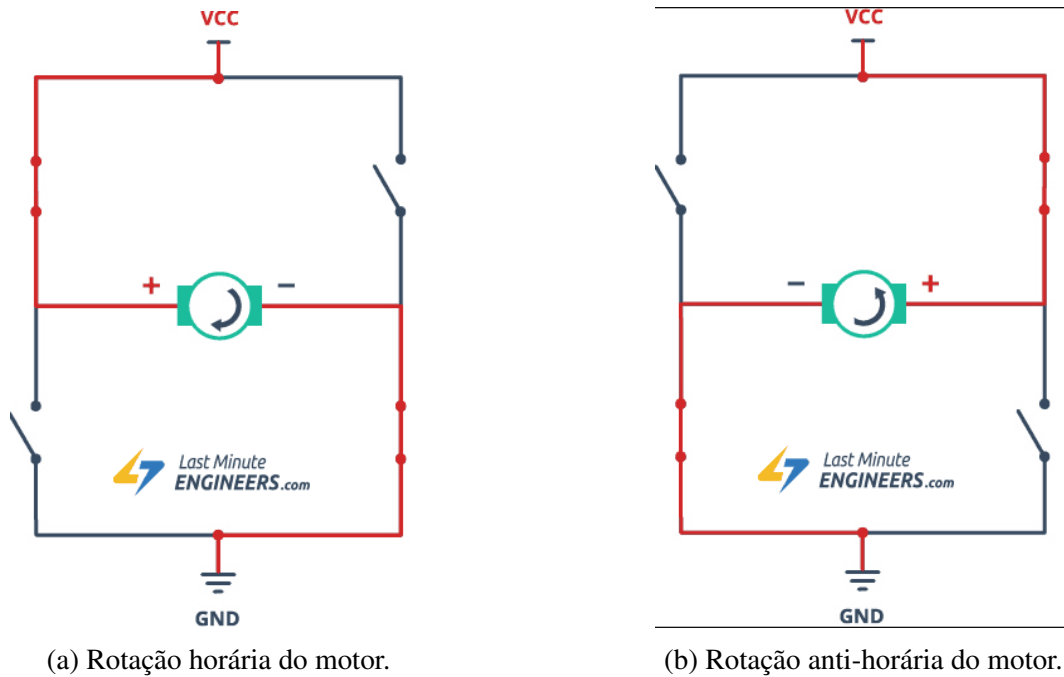


Figura 2 – Funcionamento da ponte H para diferentes sentidos de rotação.

Fonte: [LASTMINUTEENGINEERS](https://www.lastminuteengineers.com), 2024.

rotação no sentido horário ou anti-horário.

3.4 Orientação Magnética com QMC5883L

O QMC5883L é um magnetômetro digital de três eixos utilizado para medir componentes do campo magnético nos eixos X, Y e Z. Diferentemente de uma unidade inercial como o MPU6050, esse sensor não possui acelerômetro nem giroscópio, de modo que não fornece diretamente aceleração linear, velocidade angular ou temperatura útil para controle de movimento. Sua principal aplicação no contexto deste projeto é fornecer uma referência de orientação absoluta no plano horizontal, calculada a partir da relação entre as componentes magnéticas dos eixos X e Y.

O cálculo do ângulo de orientação, ou *heading*, é realizado por meio da função atan2 , aplicada às leituras convertidas para unidades proporcionais ao campo magnético. Após esse cálculo, podem ser aplicados ajustes de declinação magnética, offsets manuais de calibração e normalização angular para manter o resultado em uma faixa consistente. No protótipo, a classe customizada do QMC5883L mantém uma variável de zero angular, permitindo definir qual leitura deve ser interpretada como referência inicial do robô.

Matematicamente, o heading bruto obtido a partir do magnetômetro pode ser representado por:

$$\theta_{bruto} = \text{atan2}(B_y, B_x) \quad (3.1)$$

onde:

- B_x e B_y representam as componentes do campo magnético medidas nos eixos X e Y;
- atan2 calcula o ângulo considerando o quadrante correto da leitura;
- θ_{bruto} corresponde ao heading inicial antes dos ajustes de calibração.

Após a leitura bruta, a orientação final utilizada pelo sistema é obtida pela aplicação da declinação magnética e do offset de referência angular definido no robô:

$$\theta_{heading} = \text{mod}(\theta_{bruto} + \delta - \theta_0, 360^\circ) \quad (3.2)$$

em que δ representa a declinação magnética local e θ_0 é o zero angular configurado na calibração.

Apesar de sua utilidade como referência global, os testes práticos mostraram que o magnetômetro é sensível a interferências eletromagnéticas provenientes dos motores DC, da ponte H, da bateria e dos cabos de alimentação, além de fontes externas de ruído. Por esse motivo, o QMC5883L não foi adotado como único critério para finalizar curvas. A estratégia mais robusta implementada utiliza os encoders como fonte primária para medir a rotação executada e emprega o magnetômetro apenas como referência auxiliar quando suas leituras estão estáveis.

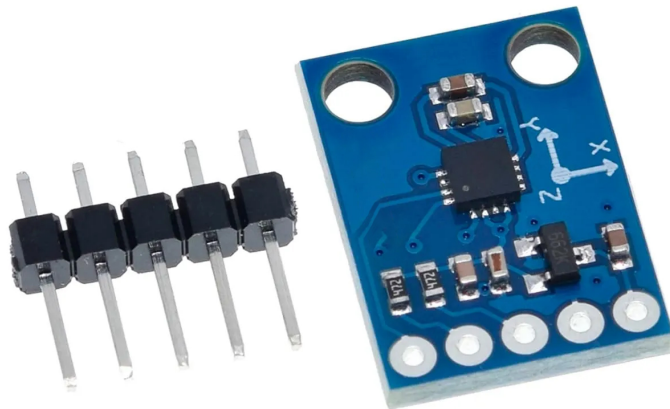


Figura 3 – Módulo magnetômetro QMC5883L.

Fonte: [CASA DA ROBOTICA, 2026](#).

3.5 Sensores Time-of-Flight: VL53L0X

Sensores do tipo *Time-of-Flight* (ToF), como o modelo VL53L0X, realizam medições de distância com base no tempo que um feixe de luz, geralmente laser infravermelho, leva para atingir um objeto e retornar ao sensor após a reflexão. Esse princípio de funcionamento permite a obtenção de leituras rápidas e com elevada precisão, alcançando uma resolução na ordem de milímetros e alcance típico de até 1,2 metros em ambientes controlados [Industries \(2024\)](#). Tais características tornam os sensores ToF especialmente adequados para aplicações que requerem resposta em tempo real, como navegação autônoma, detecção e evasão de obstáculos e mapeamento de ambientes tridimensionais.

A escolha do sensor VL53L0X neste projeto se deu, principalmente, por sua superioridade em relação a outros sensores amplamente utilizados no ensino e em projetos de prototipagem, como o HC-SR04. Enquanto o HC-SR04 utiliza ondas ultrassônicas, que estão sujeitas a variações ambientais (como temperatura e tipo de superfície do objeto), o VL53L0X opera com luz, o que lhe confere maior estabilidade, tempo de resposta mais curto e maior precisão nas medições. Dessa forma, o VL53L0X apresenta-se como uma solução mais robusta e confiável para os requisitos do sistema robótico proposto, especialmente no que se refere à detecção eficiente de obstáculos em tempo real e à construção de um mapeamento preciso do ambiente. A representação de um sensor equivalente ao utilizado no projeto está representado na Figura 4.

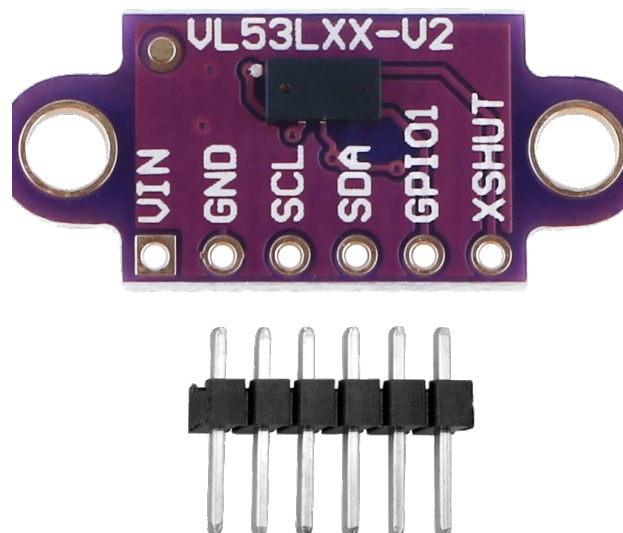


Figura 4 – Sensor VL53L0X

Fonte: [ARDUINOECIA, 2025](#).

3.6 Algoritmos PID e Odometria

O controle PID é uma das técnicas mais amplamente utilizadas em sistemas de controle automático, especialmente em aplicações embarcadas e robóticas. Sua popularidade advém da simplicidade de implementação e da eficácia no controle de processos dinâmicos. Em robótica móvel, o PID é comumente aplicado ao controle de velocidade e posição dos motores, ajustando continuamente os sinais de atuação de acordo com o erro entre o valor desejado (referência) e o valor atual medido.

Matematicamente, a saída de controle $u(t)$ do controlador PID é definida como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.3)$$

onde:

- $e(t)$ representa o erro entre a referência e a saída do sistema no instante t ;
- K_p é o ganho proporcional, que reage à magnitude do erro atual;
- K_i é o ganho integral, que corrige desvios acumulados ao longo do tempo;
- K_d é o ganho derivativo, que antecipa tendências com base na taxa de variação do erro.

A sintonização adequada desses parâmetros (K_p , K_i e K_d) é essencial para garantir estabilidade, tempo de resposta adequado e evitar oscilações ou sobrecargas no sistema, como mostrado em [Åström e Hägglund \(2006\)](#).

Complementarmente ao controle, a navegação precisa em ambientes internos exige a estimativa contínua da posição e da orientação do robô, processo conhecido como odometria. A odometria clássica baseia-se na leitura incremental dos encoders ópticos acoplados às rodas motorizadas, que fornecem a quantidade de pulsos gerados conforme as rodas giram. Com base nesses dados, é possível calcular o deslocamento linear e angular do robô por integração das variações de posição ao longo do tempo.

Considerando um robô diferencial com duas rodas, os deslocamentos linear Δs e angular $\Delta \theta$ são dados por: ([SIEGWART; NOURBAKHSH; SCARAMUZZA, 2011](#))

$$\Delta s = \frac{r}{2}(\Delta \phi_R + \Delta \phi_L) \quad (3.4)$$

$$\Delta \theta = \frac{r}{d}(\Delta \phi_R - \Delta \phi_L) \quad (3.5)$$

onde:

- r é o raio das rodas;
- d é a distância entre as rodas;
- $\Delta\phi_R$ e $\Delta\phi_L$ são os incrementos angulares das rodas direita e esquerda, respectivamente.

Essas equações permitem atualizar a posição do robô em coordenadas cartesianas (x, y) com base no deslocamento linear e na mudança de orientação, utilizando relações trigonométricas.

Apesar da sua simplicidade, a odometria baseada exclusivamente em encoders está sujeita a erros cumulativos, como deslizamentos de roda e imprecisões de leitura. Para mitigar esses erros, é comum a fusão de sensores, combinando odometria com referências externas de orientação, como o *magnetômetro QMC5883L* [Gonçalves e Silva \(2017\)](#).

Portanto, a utilização combinada do controle PID e da odometria fornece uma base robusta para a movimentação precisa e a navegação confiável do robô em ambientes internos, atendendo aos requisitos de autonomia e estabilidade do sistema proposto.

3.7 Mapeamento em Grid e Planejamento de Rotas

O mapeamento em grid é uma técnica amplamente adotada na área de robótica móvel para representação espacial de ambientes. Essa abordagem consiste na discretização do espaço contínuo em uma matriz bidimensional composta por células uniformes, também conhecidas como grades de ocupação. Cada célula da matriz representa uma região específica do ambiente e pode assumir diferentes estados, como livre, ocupada ou desconhecida, conforme as informações captadas por sensores embarcados durante o deslocamento do robô [Oliveira \(2020\)](#).

Essa representação estruturada facilita a modelagem do ambiente de forma computacionalmente tratável, permitindo a realização de tarefas como planejamento de rotas, evasão de obstáculos e atualização dinâmica do mapa conforme novas medições são realizadas. Além disso, seu formato matricial é compatível com algoritmos de busca que operam sobre grafos e redes, favorecendo a aplicação de soluções bem estabelecidas em teoria da computação.

Para a geração de trajetórias viáveis dentro desse ambiente discretizado, são utilizados algoritmos clássicos de busca e otimização de caminhos, como Dijkstra, A* (*A-star*) e métodos baseados em amostragem, como o RRT (*Rapidly-Exploring Random Tree*). Tais algoritmos visam determinar trajetórias que minimizem um determinado custo (como distância, tempo ou consumo energético), respeitando a topologia do ambiente e a presença de obstáculos. A utilização dessas técnicas é essencial para garantir a autonomia, eficiência e segurança da navegação em ambientes parcialmente ou totalmente desconhecidos.

Neste projeto, o processo de navegação e mapeamento foi desenvolvido em conjunto com o módulo de sensoriamento baseado em sensores *Time-of-Flight*, conforme descrito na Seção 3.5.

Ao invés de gerar o mapa completo em tempo de execução, o robô parte de um mapa já fornecido, sobre o qual executa a navegação e as rotinas de amostragem. Um dos objetivos era o auto-mapeamento, seguindo o modelo de [Patel \(2021\)](#), com as devidas adaptações para utilizar o sensor VL53L0X em substituição ao sensor ultrassônico HC-SR04, com o objetivo de obter maior precisão e menor latência nas medições de distância.

Para a implementação prática, o ambiente foi modelado de forma simplificada, assumindo-se as seguintes definições geométricas:

- **Dimensões do robô:** representado por um retângulo de largura W e comprimento L ;
- **Dimensões do ambiente:** representadas por uma área retangular de largura R_W e comprimento R_L ;
- **Tamanho da matriz de mapeamento:** definido como $m \times n$, onde $m = \frac{R_L}{L}$ e $n = \frac{R_W}{W}$, correspondendo ao número de células percorríveis no eixo vertical e horizontal, respectivamente.

Essa modelagem permite a construção de uma matriz binária que identifica as áreas acessíveis e intransponíveis, conforme os dados sensoriais obtidos ao longo da movimentação do robô. A Figura 5, extraída da proposta original de [Patel \(2021\)](#), ilustra esse conceito de discretização do espaço.

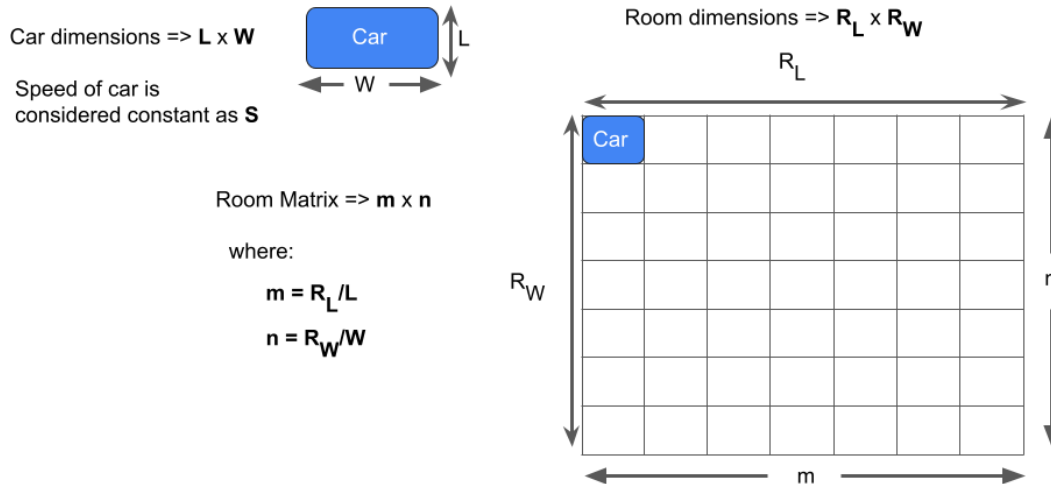


Figura 5 – Exemplo simplificado de mapeamento em grid.

No contexto experimental deste trabalho, essa representação foi adaptada para um mapa discreto de dimensões 7×7 , utilizado como ambiente controlado para validação inicial dos procedimentos de navegação, localização e detecção de obstáculos. Cada célula da matriz representa

uma área física de aproximadamente $30\text{ cm} \times 30\text{ cm}$, de modo que os deslocamentos do robô são tratados como movimentos entre posições discretas do grid. A Figura 6 apresenta o mapa utilizado nessa etapa, no qual os quadrados rubricados indicam as células ocupadas ou intransponíveis, enquanto as demais células representam regiões livres para navegação. A posição inicial do robô é representada nas coordenadas (1, 1) por um círculo azul acompanhado de uma seta, sendo esta responsável por indicar a pose do robô, isto é, sua posição e orientação no mapa.

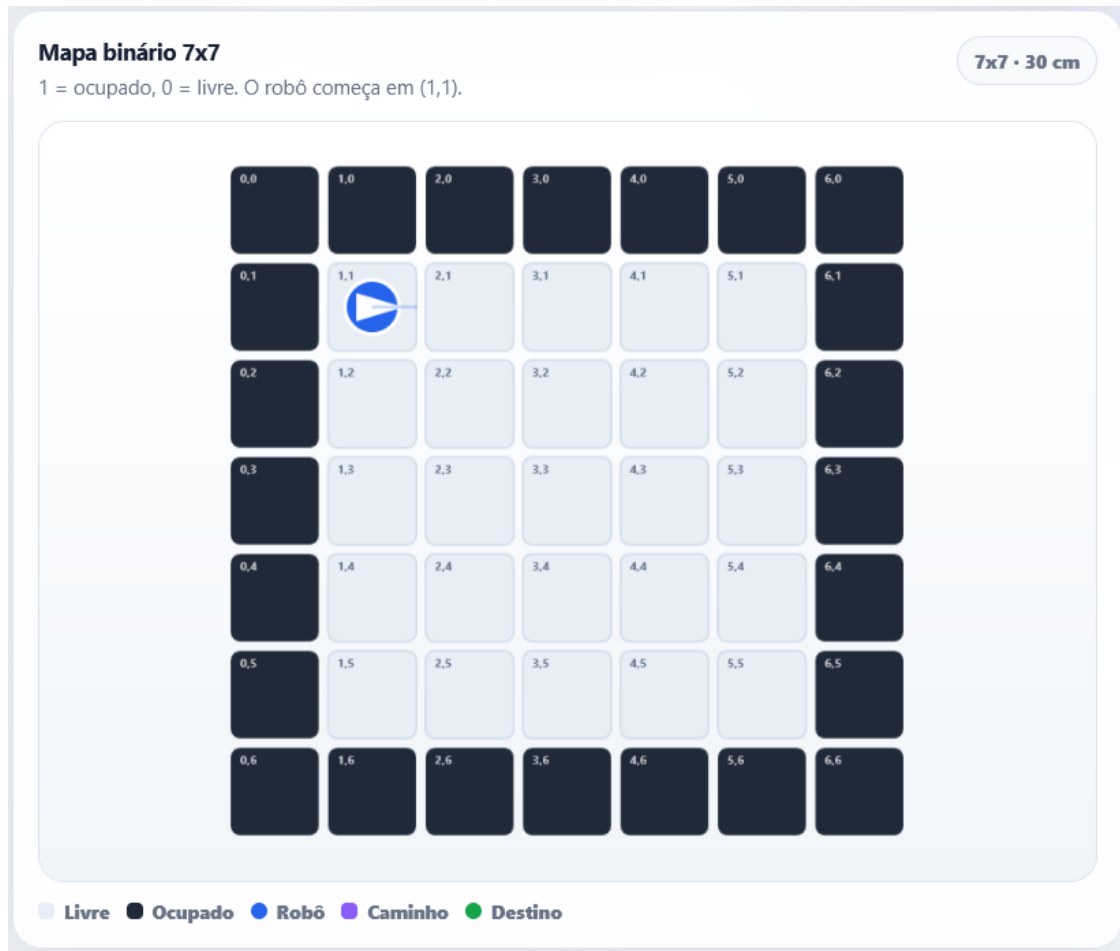


Figura 6 – Mapa 7x7 utilizado para validação da navegação, com células de $30\text{ cm} \times 30\text{ cm}$, células ocupadas rubricadas e representação da pose inicial do robô em (1, 1).

Com a grade definida, a estratégia de mapeamento baseia-se na movimentação sistemática do robô sobre as células da matriz, percorrendo coordenadas discretas (x, y) e atualizando o estado de cada célula com base nos dados coletados em tempo real. Na fase experimental, o mapa utilizado foi o grid 7x7 apresentado na Figura 6, fornecido previamente como base para validação da navegação e da amostragem. Para a amostragem final, esse mapa foi expandido para proporções maiores, de forma a representar melhor o ambiente e permitir testes mais próximos de cenários reais.

A Figura 7 exemplifica um padrão de percurso planejado para cobrir integralmente todas as células da matriz. Tal trajetória garante a coleta de dados em todas as áreas do espaço repre-

sentado, permitindo a construção do mapa de ocupação fornecido. Contudo, o modelo inspirado em Patel (2021) não permite um grande detalhamento do local caso haja um objeto no meio do caminho, mas dado objeto seja apenas local, como cadeiras, bancos e outras mobílias que não necessariamente ficam em um canto da sala. No entanto, para o escopo deste trabalho, cujo objetivo é validar a navegação autônoma em ambientes estruturados (como corredores e salas com obstáculos predominantemente periféricos), este modelo de mapa de ocupação é considerado suficiente e adequado.

Mapping

Mapping of the room is done as shown in this figure. It starts with the top right corner of the room.

Readings are saved in form of a 2D array with **n rows** and **3 columns**.

Readings are saved up to **n** horizontal iterations with the middle column of the array of horizontal distance and the other two for extra space on the side. It will be explained later.

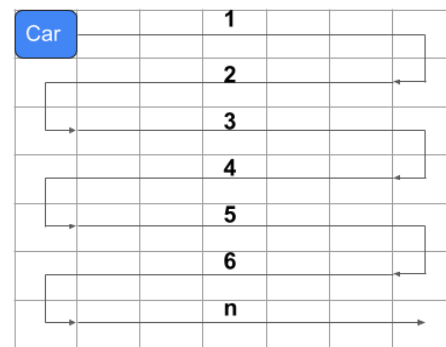


Figura 7 – Exemplo de trajetória do robô durante o processo de mapeamento.

Para a realização da locomoção autônoma do robô dentro do ambiente mapeado, é necessário fornecer uma coordenada de destino, expressa em termos da matriz gerada durante o processo de discretização do espaço. Essa coordenada representa a posição final desejada no grid de ocupação, e deve corresponder a uma célula classificada como livre e acessível, conforme os dados obtidos na etapa de mapeamento.

A execução do deslocamento até essa posição depende do conhecimento da posição atual do robô na matriz. Três hipóteses são consideradas válidas para determinar essa posição com segurança:

- **Hipótese 1 — Localização por trajetória anterior:** o robô já percorreu previamente parte do ambiente e, portanto, sua posição atual pode ser estimada com base na trajetória registrada, utilizando dados de odometria e sensoriamento contínuo. Essa abordagem assume que não houve perdas significativas de precisão acumulada, como deslizamentos ou ruídos nos sensores (estes sendo reduzidos pelo PID e métodos mencionados na Seção 3.6).

- **Hipótese 2 — Ponto de partida fixo:** o robô inicia sua operação a partir de uma posição conhecida e previamente definida, geralmente no canto superior esquerdo da matriz, correspondente à coordenada (1, 1). Essa configuração é típica de ambientes controlados, como laboratórios, nos quais a posição inicial pode ser garantida fisicamente.
- **Hipótese 3 — Localização informada manualmente:** a posição atual do robô é fornecida manualmente por meio de uma interface externa, seja via sistema de monitoramento, interface gráfica ou inserção direta na interface web embarcada no robô. Essa abordagem é útil em cenários nos quais a posição inicial é identificável visualmente ou quando o sistema é reiniciado em um ponto intermediário do ambiente, contudo, esta hipótese depende do usuário saber identificar com certeza a localização que o robô está.

Com a posição de origem e o destino definidos dentro da matriz, um algoritmo de planejamento de rotas pode ser aplicado, como o A* ou Dijkstra, para determinar o melhor caminho possível entre os dois pontos, levando em consideração os obstáculos registrados e a topologia do ambiente. O caminho gerado é, então, convertido em uma sequência de comandos de movimento (direções e distâncias) que são enviados ao sistema de controle do robô para execução em tempo real.

Essa abordagem garante que o robô seja capaz de se deslocar de forma autônoma e eficiente dentro do ambiente previamente mapeado, utilizando a representação discreta do espaço como base para a navegação segura e orientada por rotas. Para lidar com a natureza dinâmica dos ambientes reais, que podem conter obstáculos não mapeados ou móveis (como pessoas), a estratégia de navegação adotada é de natureza híbrida. Ela combina um **planejador de caminho global** com um **controlador reativo local**.

- **Planejador Global:** Utilizando o mapa de grid estático gerado previamente, um algoritmo de busca como o A* ou Dijkstra calcula a rota ótima do ponto de partida ao destino. Essa rota serve como um guia macro para a trajetória do robô.
- **Controlador Reativo Local:** Durante a execução da trajetória, o sensor VL53L0X monitora continuamente o caminho imediato à frente do robô. Caso um obstáculo inesperado seja detectado, o controlador reativo assume prioridade, pausando ou alterando a trajetória localmente para evitar a colisão, para então tentar retornar ao caminho global planejado.

Essa arquitetura de duas camadas confere ao robô a capacidade de seguir um plano de longo prazo enquanto reage de forma inteligente e segura a eventos imprevistos, aumentando significativamente sua autonomia e robustez.

4 Materiais e Métodos

Este capítulo descreve, de maneira detalhada, os materiais empregados e os procedimentos metodológicos adotados para o desenvolvimento do protótipo de robô autônomo proposto. A escolha criteriosa dos componentes eletrônicos, sensores e atuadores foi orientada pelas necessidades do sistema, levando em consideração fatores como precisão, custo, disponibilidade e compatibilidade com a plataforma de desenvolvimento utilizada.

São apresentados o ambiente de desenvolvimento de software, as bibliotecas utilizadas, os recursos computacionais embarcados, bem como a lógica de controle implementada no microcontrolador. A metodologia abrange também a integração dos sensores, o controle dos motores, o modelo de mapeamento em grid e a abordagem utilizada para planejamento de rotas.

Do ponto de vista físico, o robô é constituído por uma base de pequeno porte sobre a qual foram fixados os módulos eletrônicos, sensores e motores. O sistema de locomoção é composto por quatro motores DC organizados em pares esquerdo e direito, acionados por meio de um módulo de ponte H baseado no L298N. A orientação auxiliar é obtida por um magnetômetro QMC5883L, enquanto a medição de deslocamento e rotação utiliza encoders ópticos. A detecção de obstáculos frontais é realizada com um sensor de distância do tipo *Time-of-Flight*, modelo VL53L0X. O controle do sistema e a comunicação com o usuário são realizados via microcontrolador ESP32 com interface web embarcada, acessível por meio de rede Wi-Fi local.

4.1 Arquitetura do Sistema e Prototipagem de Hardware

A arquitetura do sistema foi definida com o objetivo de integrar, de maneira modular, os componentes de sensoriamento, processamento, atuação, alimentação elétrica e interface com o usuário necessários para o funcionamento autônomo do robô móvel. Essa organização permite compreender o sistema não apenas como um conjunto de componentes eletrônicos isolados, mas como uma estrutura embarcada integrada, na qual os dados coletados pelos sensores são processados pelo microcontrolador e convertidos em comandos de controle para os motores.

A Figura 8 apresenta a arquitetura geral adotada no projeto, destacando os principais módulos utilizados e os fluxos de dados, controle e alimentação elétrica entre eles.

Conforme ilustrado na Figura 8, o ESP32 atua como unidade central de processamento do robô. Esse microcontrolador é responsável pela aquisição dos dados dos sensores, pelo tratamento das leituras recebidas, pela execução da lógica de navegação, pelo controle dos motores e pela comunicação com a interface web. A escolha do ESP32 está relacionada à sua capacidade de processamento, à disponibilidade de interfaces de comunicação, ao suporte a interrupções para lei-

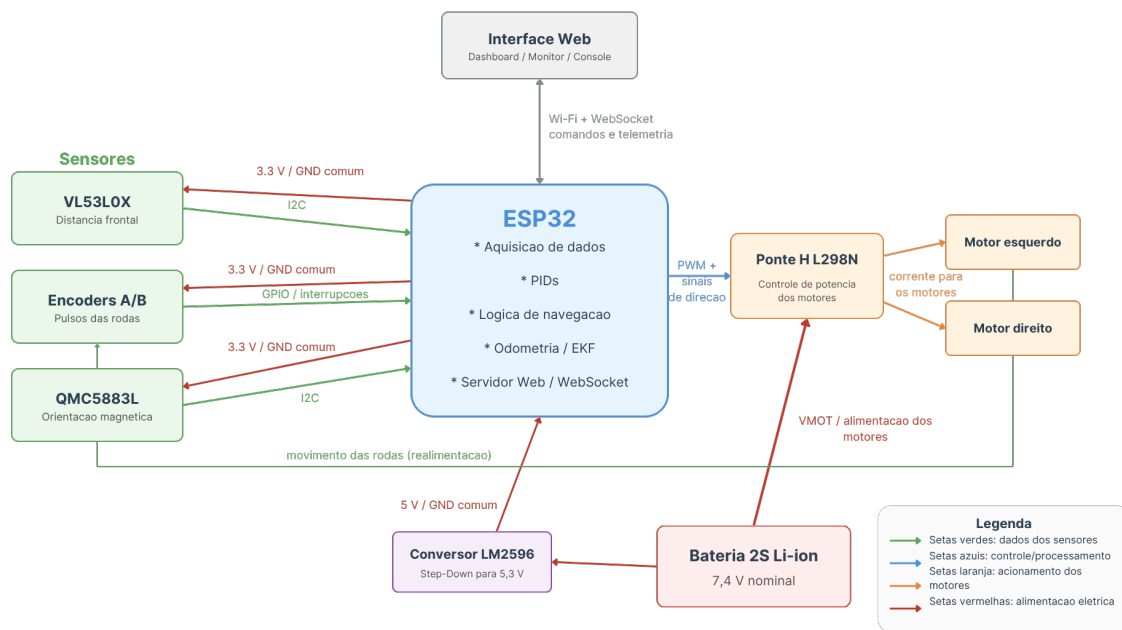


Figura 8 – Arquitetura geral do sistema robótico embarcado.

tura dos encoders e à conectividade Wi-Fi integrada, características discutidas na fundamentação teórica e aproveitadas diretamente na implementação do sistema.

O módulo de sensoriamento é composto principalmente pelos encoders ópticos, pelo sensor de distância VL53L0X e pelo magnetômetro QMC5883L. Os encoders fornecem pulsos associados à rotação das rodas, permitindo estimar o deslocamento do robô, calcular a odometria e controlar os movimentos lineares e angulares. O sensor VL53L0X, baseado em tecnologia *Time-of-Flight*, é utilizado para medir a distância frontal e identificar obstáculos durante a navegação. Já o QMC5883L fornece uma referência auxiliar de orientação magnética no plano horizontal, sendo empregado para estimativa do *heading* quando suas leituras apresentam estabilidade suficiente.

A partir das informações sensoriais, o ESP32 executa rotinas de controle e decisão, incluindo controladores PID, lógica de navegação em grid, planejamento de movimento entre células, monitoramento de obstáculos e atualização da posição estimada do robô. Essas informações são utilizadas para gerar sinais de controle enviados ao módulo Ponte H L298N. A Ponte H recebe sinais digitais e PWM do ESP32 e realiza o acionamento elétrico dos motores de corrente contínua, permitindo o controle independente dos lados esquerdo e direito do robô.

A estrutura física do robô é baseada em uma plataforma de quatro rodas acionadas por motores de corrente contínua. Cada lado possui dois motores conectados em paralelo, formando dois conjuntos de tração: um conjunto esquerdo e um conjunto direito. Essa configuração permite a locomoção diferencial do robô, possibilitando movimentos de avanço, ré e rotação sobre o próprio eixo. Os motores frontais são equipados com encoders, utilizados como principal fonte de realimentação para o controle de deslocamento e rotação.

O sistema de alimentação foi organizado de forma a separar a alimentação de potência dos motores da alimentação lógica dos circuitos de controle e sensoriamento. A bateria Li-ion 2S, com tensão nominal de 7,4 V, alimenta diretamente a Ponte H L298N, responsável pelo fornecimento de corrente aos motores. Em paralelo, a mesma bateria alimenta um conversor regulador LM2596, responsável por fornecer uma tensão regulada de 5 V para o ESP32 e os sensores. Todos os módulos compartilham uma referência comum de GND, condição necessária para o funcionamento adequado dos sinais de controle e comunicação entre os componentes.

A interface com o usuário é realizada por meio de uma interface web embarcada no ESP32, acessível via rede Wi-Fi local. Essa interface permite enviar comandos ao robô, selecionar destinos no mapa, acompanhar a telemetria, visualizar leituras dos sensores, observar a pose estimada do robô e consultar mensagens de registro do sistema. A troca de informações entre a interface e o microcontrolador ocorre por meio de WebSockets e mensagens em formato JSON, permitindo comunicação bidirecional em tempo real.

Dessa forma, a arquitetura adotada estabelece um ciclo de realimentação entre sensoriamento, processamento e atuação. Os sensores informam o estado físico do robô e do ambiente, o ESP32 processa essas informações e define as ações de controle, a Ponte H converte os comandos em acionamento elétrico dos motores, e a interface web permite o monitoramento e a interação com o sistema durante os testes.

4.1.1 Prototipagem e Ajustes de Hardware

Após a definição da arquitetura geral, os componentes foram organizados em uma montagem de prototipagem para validar, de forma prática, a integração entre os módulos apresentados na fundamentação teórica. Essa etapa permitiu verificar a compatibilidade entre o ESP32, a Ponte H L298N, os motores DC, os encoders, o sensor VL53L0X, o magnetômetro QMC5883L, o conversor LM2596 e o sistema de alimentação por bateria.

A Figura 9 apresenta a organização física revisada dos componentes em ambiente de prototipação. Essa representação foi utilizada como referência para a montagem do protótipo, evidenciando a disposição dos módulos principais, das conexões de alimentação e dos sinais de controle.

Durante os testes de integração, foram realizados ajustes na disposição dos componentes e no roteamento das conexões, buscando reduzir desconexões, facilitar a manutenção e melhorar a estabilidade elétrica do sistema durante a movimentação do robô. A separação entre a alimentação dos motores e a alimentação do ESP32 e sensores foi um ponto importante dessa etapa, pois os motores podem introduzir ruídos e quedas de tensão capazes de afetar o funcionamento dos circuitos lógicos.

Também foram observadas limitações práticas associadas ao uso do magnetômetro QMC5883L. Embora o sensor tenha sido incorporado como referência auxiliar de orientação, os testes indica-

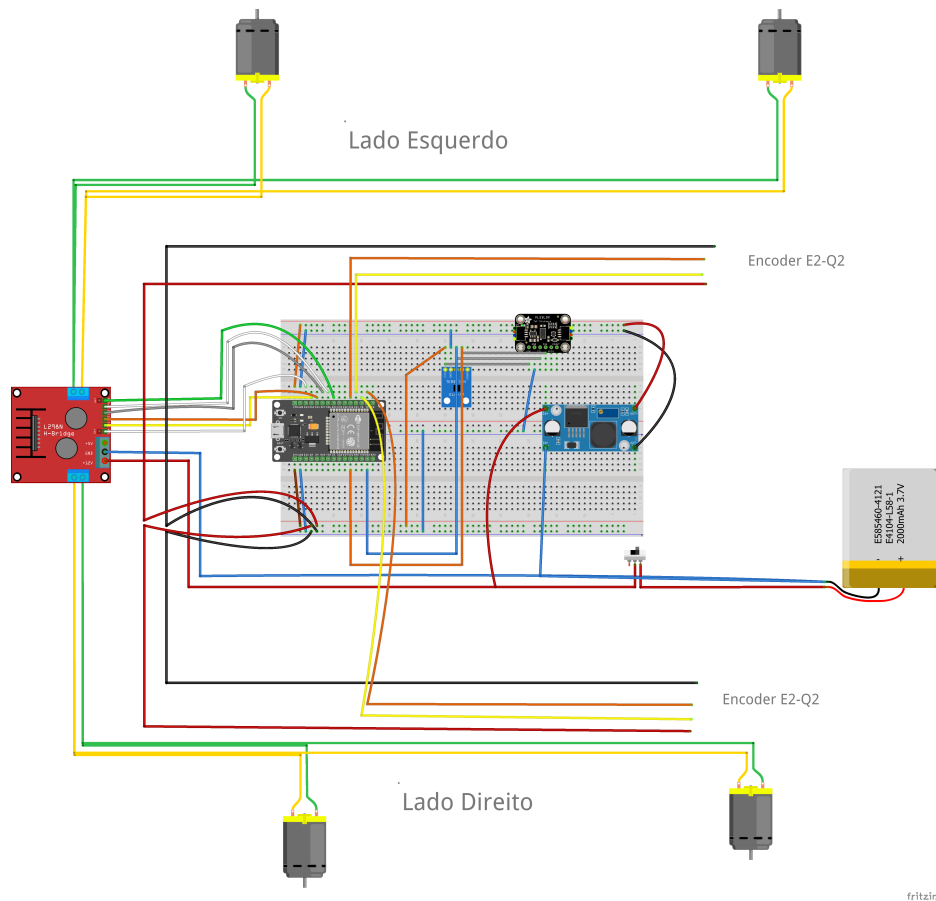


Figura 9 – Organização revisada dos componentes utilizados no protótipo do robô móvel. Modelo desenvolvido com auxílio da ferramenta [Fritzing.org](https://www.fritzing.org/).

ram que suas leituras podem sofrer interferências causadas por motores, correntes elétricas, fios de alimentação e componentes metálicos próximos. Por esse motivo, os encoders permaneceram como principal fonte de informação para controle de deslocamento e rotação, enquanto o QMC5883L foi tratado como uma referência complementar, utilizada apenas quando suas leituras apresentavam estabilidade suficiente.

A etapa de prototipagem, portanto, permitiu consolidar a arquitetura de hardware utilizada no projeto, aproximando os conceitos discutidos na fundamentação teórica da implementação física do robô. Além disso, os ajustes realizados nessa fase contribuíram diretamente para a execução dos testes de navegação em grid, deslocamento entre células, detecção de obstáculos, controle por encoders e monitoramento remoto por interface web.

4.2 Ambiente de Desenvolvimento

O desenvolvimento do *firmware* embarcado do robô autônomo foi realizado no *Visual Studio Code*, utilizando a extensão PlatformIO e o *framework* Arduino para ESP32. Essa escolha permitiu organizar o projeto em uma estrutura modular, com separação entre código-fonte, bibliotecas

customizadas, arquivos de interface web e configurações de compilação.

O uso do PlatformIO facilitou o gerenciamento das dependências, a configuração da placa ESP32, a compilação do projeto e o envio dos arquivos estáticos da interface para a memória flash do microcontrolador. Como a interface web é composta por arquivos HTML, CSS e JavaScript, o suporte a sistemas de arquivos embarcados, como SPIFFS, tornou-se parte importante do fluxo de desenvolvimento.

Além da organização do código, o ambiente também favoreceu a depuração incremental. Durante o desenvolvimento, logs via serial e via interface web foram utilizados para acompanhar leituras dos encoders, estado do magnetômetro, detecção de obstáculos, planejamento de rota, execução de células e calibração de curvas.

4.2.1 Execução concorrente com FreeRTOS

Embora o desenvolvimento do *firmware* tenha sido realizado com o *framework* Arduino para ESP32, a execução das rotinas no microcontrolador ocorre sobre a infraestrutura do FreeRTOS, conforme discutido na Seção 3.2. No projeto, essa característica foi explorada para organizar partes do sistema em tarefas concorrentes, separando rotinas com responsabilidades distintas, como leitura de sensores, atualização de telemetria, comunicação com a interface web e controle de movimentação.

Essa organização foi importante para manter a responsividade do robô durante a navegação. Em um sistema móvel autônomo, o controle dos motores e a leitura dos sensores não podem depender exclusivamente de uma execução sequencial bloqueante, pois atrasos em uma rotina poderiam comprometer a detecção de obstáculos, o envio de dados à interface ou a atualização da posição estimada. Assim, a divisão em tarefas contribuiu para que o sistema continuasse processando informações de sensoriamento e comunicação enquanto executava comandos de movimento.

No contexto deste trabalho, o uso do FreeRTOS favoreceu a separação entre aquisição de dados, controle de baixo nível, comunicação com o usuário e registro de eventos. Essa abordagem também facilitou os testes incrementais do sistema, uma vez que permitiu isolar responsabilidades do *firmware* e acompanhar o comportamento de cada módulo durante a execução experimental.

4.2.2 Bibliotecas Utilizadas

Para a implementação das funcionalidades do sistema embarcado, foram utilizadas bibliotecas tanto nativas quanto de terceiros. Essas bibliotecas atuaram na abstração de protocolos de comunicação, controle de hardware, gerenciamento de rede e suporte à interface web. A Tabela 1 apresenta as principais bibliotecas utilizadas e suas respectivas finalidades.

Durante as primeiras versões do sistema, foi avaliada a utilização de resolução local de nome por DNS/mDNS, de modo que a interface web pudesse ser acessada por um endereço ami-

Tabela 1 – Bibliotecas externas e oficiais utilizadas no projeto

Biblioteca	Finalidade
Wire.h	Comunicação I2C com sensores como QMC5883L e VL53L0X.
Arduino.h	Funções básicas de controle de pinos, temporização e compatibilidade com o ecossistema Arduino.
FreeRTOS	Infraestrutura de execução concorrente disponível no ESP32, utilizada para criação e organização de tarefas do sistema embarcado.
math.h	Cálculos matemáticos, incluindo trigonometria e manipulação de ponto flutuante.
ArduinoJson.h	Criação e interpretação de objetos JSON para comunicação com a interface web.
WiFi.h	Conexão do ESP32 à rede Wi-Fi local.
ESPAsyncWebServer.h	Criação de servidor web assíncrono, com suporte a múltiplas conexões simultâneas.
SPIFFS.h	Leitura e escrita dos arquivos da interface web no sistema de arquivos embarcado.
Adafruit_VL53L0X.h	Controle do sensor de distância VL53L0X.
Adafruit_Sensor.h	Interface comum para sensores Adafruit.
QMC5883L_Custom	Biblioteca própria para configuração, leitura e filtragem do magnetômetro QMC5883L.

gável na rede local. Entretanto, nos testes práticos, esse recurso não se mostrou suficientemente útil ou consistente em diferentes cenários de conexão, especialmente quando o acesso era realizado por meio de rede compartilhada pelo roteador do celular. Por esse motivo, a abordagem foi removida da versão final, optando-se pelo acesso direto ao endereço IP atribuído ao ESP32 pelo roteador. Essa decisão simplificou a conexão com a interface web e reduziu uma dependência adicional da camada de rede.

4.2.3 Bibliotecas Customizadas

A fim de modularizar o sistema, reduzir a complexidade do código-fonte principal e facilitar a manutenção e reuso de funcionalidades, diversas bibliotecas personalizadas foram desenvolvidas especificamente para o projeto. Essas bibliotecas encapsulam funcionalidades críticas como controle de motores, leitura de sensores, filtragem de sinais e gerenciamento de energia. A seguir, são listadas as bibliotecas customizadas implementadas:

- **PID:** Implementa o algoritmo de controle Proporcional–Integral–Derivativo, permitindo múltiplas instâncias com parâmetros distintos. Essencial para o controle de velocidade dos motores com base em dados de odometria.
- **Filtros e validações:** Realizam suavização e validação de leituras, incluindo rejeição de

amostras anômalas do QMC5883L e de saltos incompatíveis dos encoders.

- **Encoder:** Abstrai a leitura dos encoders das rodas, permitindo o cálculo da velocidade, do deslocamento linear e da rotação estimada durante curvas.
- **Monitoramento e logs:** Agrupa rotinas de diagnóstico utilizadas para registrar telemetria, estado de navegação, erros de sensores e eventos relevantes para calibração.
- **VL53L0X (custom):** Estende a biblioteca oficial com lógica adicional para detecção de obstáculos com limiares configuráveis.
- **QMC5883L (custom):** Integra a configuração do magnetômetro, a leitura dos eixos X, Y e Z, o cálculo de *heading*, a aplicação de offset angular e a rejeição de leituras instáveis.
- **Motor:** Controla individualmente cada motor, centralizando a lógica de PWM, leitura de encoder e resposta ao PID.
- **PonteH:** Biblioteca de alto nível responsável pelo controle simultâneo dos motores do robô. Gerencia comandos de movimento, como deslocamento linear, curvas e manobras de rotação no próprio eixo. Também integra informações de sensores para ajuste de trajetória e comunicação em tempo real com a interface web.

A combinação dessas bibliotecas permitiu uma separação clara entre as responsabilidades de cada módulo do sistema, promovendo um desenvolvimento mais organizado, escalável e de fácil manutenção.

4.3 Estratégias de Controle e Navegação

A navegação autônoma do robô é composta por um conjunto de estratégias interligadas que envolvem sensoramento, controle de movimento, estimativa de posição e planejamento de trajetória. O sistema foi projetado para operar de forma reativa e deliberativa, ou seja, combinando a coleta contínua de dados do ambiente com decisões de alto nível baseadas em um mapa interno.

4.3.1 Odometria e Estimativa de Posição

A estimativa de deslocamento do robô é realizada por meio da odometria diferencial, utilizando a contagem de pulsos dos encoders ópticos instalados nas rodas frontais. Esses dados são processados para determinar a distância percorrida por cada lado do robô e a variação angular ao longo do tempo. Como o robô utiliza uma configuração de tração diferencial, na qual os lados esquerdo e direito podem ser acionados de forma independente, a pose do robô pode ser estimada a partir da diferença entre os deslocamentos das rodas.

Inicialmente, os pulsos acumulados pelos encoders são convertidos em deslocamento linear. Considerando uma roda de raio r , uma resolução de encoder de N pulsos por volta e variações de contagem Δp_L e Δp_R para as rodas esquerda e direita, respectivamente, os deslocamentos lineares de cada lado podem ser estimados por:

$$\Delta s_L = \frac{2\pi r}{N} \Delta p_L \quad (4.1)$$

$$\Delta s_R = \frac{2\pi r}{N} \Delta p_R \quad (4.2)$$

em que Δs_L representa o deslocamento linear do lado esquerdo e Δs_R representa o deslocamento linear do lado direito. A partir desses valores, calcula-se o deslocamento médio do robô e a variação angular aproximada:

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2} \quad (4.3)$$

$$\Delta \theta = \frac{\Delta s_R - \Delta s_L}{b} \quad (4.4)$$

em que b representa a distância entre as rodas ou, de forma equivalente, a largura efetiva entre os pontos de contato dos lados esquerdo e direito do robô. Com esses valores, a pose do robô no instante k pode ser atualizada a partir da pose anterior $(x_{k-1}, y_{k-1}, \theta_{k-1})$, conforme:

$$x_k = x_{k-1} + \Delta s \cos \left(\theta_{k-1} + \frac{\Delta \theta}{2} \right) \quad (4.5)$$

$$y_k = y_{k-1} + \Delta s \sin \left(\theta_{k-1} + \frac{\Delta \theta}{2} \right) \quad (4.6)$$

$$\theta_k = \theta_{k-1} + \Delta \theta \quad (4.7)$$

Essa formulação permite atualizar incrementalmente a posição e a orientação do robô a cada nova leitura dos encoders. O termo θ representa a orientação do robô no plano, enquanto x e y indicam sua posição estimada em relação ao referencial adotado no mapa. No contexto deste trabalho, essa estimativa é utilizada tanto para acompanhar o deslocamento físico do robô quanto para relacionar sua posição contínua com as células discretas do mapa em grid.

Para a aplicação prática dessas equações, é necessário definir alguns parâmetros físicos do protótipo. Esses parâmetros relacionam a movimentação real do robô com os valores obtidos pelos encoders e, portanto, influenciam diretamente a estimativa de pose. No presente trabalho, foram considerados os seguintes pontos de medição:

- **Raio da roda ($r = 325 \text{ mm}$):** corresponde à distância entre o centro da roda e sua extremidade externa. Esse valor é utilizado para converter a rotação medida pelo encoder em deslocamento linear.
- **Diâmetro da roda ($D = 650 \text{ mm}$):** corresponde à distância total entre duas extremidades opostas da roda, passando pelo centro. Como $D = 2r$, esse valor também pode ser utilizado para determinar o perímetro da roda, dado por $C = \pi D$.
- **Distância entre rodas ou largura efetiva da base ($b = 150 \text{ mm}$):** corresponde à distância entre os pontos de contato das rodas esquerda e direita com o solo. Esse parâmetro é utilizado no cálculo da variação angular $\Delta\theta$, sendo essencial para estimar corretamente as rotações do robô.
- **Comprimento total do robô ($L = 26 \text{ cm}$):** corresponde à dimensão longitudinal do chassi, medida da extremidade frontal até a extremidade traseira. Essa medida é relevante para verificar se o robô cabe dentro de uma célula do grid e para avaliar sua margem de manobra durante a navegação.
- **Largura total do robô ($W = 15 \text{ cm}$):** corresponde à dimensão transversal do chassi, medida de uma lateral à outra. Essa medida permite comparar a ocupação física do robô com o tamanho das células do mapa, definidas neste trabalho como $30 \text{ cm} \times 30 \text{ cm}$.
- **Distância entre o sensor frontal e a dianteira do robô ($d_s = 5 \text{ cm}$):** corresponde ao posicionamento do sensor VL53L0X em relação à extremidade frontal do chassi. Esse valor é importante para interpretar corretamente as leituras de distância, pois o sensor mede a distância a partir de sua própria posição, e não necessariamente a partir da frente física do robô.
- **Altura aproximada do sensor frontal ($h_s = 7 \text{ cm}$):** corresponde à altura do VL53L0X em relação ao solo. Essa medida influencia a detecção de obstáculos, principalmente quando os objetos possuem alturas diferentes ou não interceptam diretamente o feixe de medição do sensor.
- **Resolução dos encoders ($N = 20 \text{ pulsos/rev}$):** corresponde ao número de pulsos gerados por volta da roda ou do eixo monitorado. Esse valor é necessário para converter a contagem de pulsos em deslocamento linear.

Essas dimensões permitem relacionar o modelo matemático de odometria com a geometria real do robô. No caso específico deste projeto, a comparação entre o tamanho físico do protótipo e as células de $30 \text{ cm} \times 30 \text{ cm}$ também é importante para justificar a escolha do grid experimental, uma vez que o robô precisa se deslocar entre células consecutivas mantendo margem suficiente para correção de trajetória e detecção de obstáculos.

A odometria, apesar de eficiente em curtos intervalos de tempo, sofre com acúmulo de erro ao longo de percursos maiores. Esse erro pode ser causado por deslizamento das rodas, folgas mecânicas, irregularidades do piso, diferenças entre motores e ruídos nas leituras dos encoders. No protótipo atual, os encoders são utilizados como fonte primária para medição de deslocamento e controle de rotação, pois apresentaram maior consistência prática para a execução dos movimentos entre células do grid.

O magnetômetro QMC5883L é empregado como referência auxiliar de orientação, porém suas leituras são filtradas e rejeitadas quando apresentam saltos incompatíveis com a movimentação esperada. Essa estratégia evita que interferências magnéticas causadas por motores, correntes elétricas ou componentes metálicos próximos afetem diretamente a estimativa da pose do robô. Dessa forma, a odometria baseada nos encoders permanece como base principal da localização, enquanto o sensor magnético atua apenas como informação complementar quando suas medições são consideradas estáveis.

4.3.2 Controle de Movimento via PID

O controle de movimento do robô é implementado por meio de controladores PID, utilizados para reduzir o erro entre uma grandeza de referência e a grandeza efetivamente medida pelo sistema. No protótipo desenvolvido, o controle foi organizado em duas malhas principais: uma malha de velocidade, associada ao controle dos motores a partir dos encoders, e uma malha de direção, associada à correção da orientação do robô durante deslocamentos lineares e rotações.

A malha de velocidade atua sobre cada lado do robô de forma independente. Com base nos dados dos encoders, o sistema estima a velocidade angular das rodas e ajusta dinamicamente a largura de pulso (PWM) aplicada aos motores por meio da Ponte H L298N. Essa estratégia permite compensar diferenças mecânicas entre os motores, variações de carga, atrito e pequenas assimetrias na montagem. A Figura 10 apresenta o diagrama simplificado dessa malha de controle.

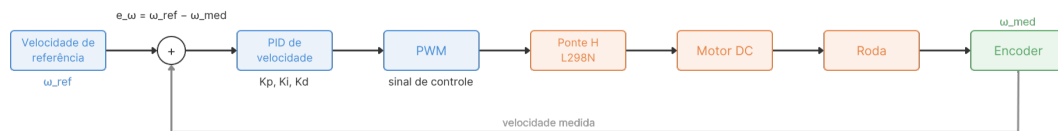


Figura 10 – Diagrama simplificado da malha de controle de velocidade dos motores.

Para cada motor, o erro de velocidade é definido como a diferença entre a velocidade angular de referência e a velocidade angular medida pelos encoders:

$$e_{\omega}(t) = \omega_{\text{ref}}(t) - \omega_{\text{med}}(t) \quad (4.8)$$

em que $\omega_{\text{ref}}(t)$ representa a velocidade angular desejada e $\omega_{\text{med}}(t)$ representa a velocidade angular estimada a partir dos pulsos dos encoders. A ação de controle PID para a malha de velocidade é dada por:

$$u_{\omega}(t) = K_{p\omega}e_{\omega}(t) + K_{i\omega} \int e_{\omega}(t) dt + K_{d\omega} \frac{de_{\omega}(t)}{dt} \quad (4.9)$$

em que $K_{p\omega}$, $K_{i\omega}$ e $K_{d\omega}$ representam, respectivamente, os ganhos proporcional, integral e derivativo da malha de velocidade. O sinal $u_{\omega}(t)$ é convertido em um valor de PWM aplicado ao motor correspondente.

Além da malha de velocidade, o sistema utiliza uma malha de direção para corrigir desvios de orientação. Essa malha compara a orientação de referência do robô com a orientação estimada durante o movimento, produzindo uma correção angular que é distribuída entre os lados esquerdo e direito. A Figura 11 apresenta o diagrama simplificado dessa malha.

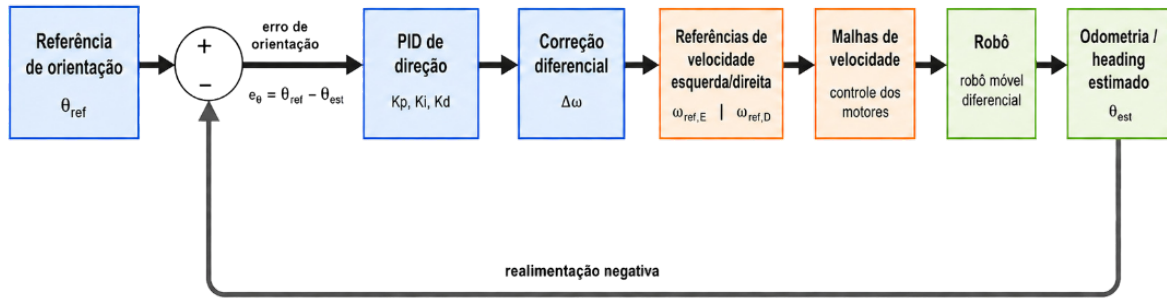


Figura 11 – Diagrama simplificado da malha de controle de direção do robô.

O erro angular é calculado a partir da diferença entre a orientação desejada e a orientação estimada do robô:

$$e_{\theta}(t) = \theta_{\text{ref}}(t) - \theta_{\text{med}}(t) \quad (4.10)$$

Como a orientação é uma grandeza angular, esse erro deve ser normalizado para evitar ambiguidades entre ângulos equivalentes. Assim, o erro utilizado pelo controlador é dado por:

$$e_{\theta}^*(t) = \text{norm}(\theta_{\text{ref}}(t) - \theta_{\text{med}}(t)) \quad (4.11)$$

em que $\text{norm}(\cdot)$ representa a normalização do erro angular para o intervalo $[-\pi, \pi]$. A ação de controle da malha de direção é definida por:

$$u_\theta(t) = K_{p\theta}e_\theta^*(t) + K_{i\theta} \int e_\theta^*(t) dt + K_{d\theta} \frac{de_\theta^*(t)}{dt} \quad (4.12)$$

em que $K_{p\theta}$, $K_{i\theta}$ e $K_{d\theta}$ representam os ganhos proporcional, integral e derivativo da malha de direção. O termo $u_\theta(t)$ corresponde à correção angular aplicada ao movimento do robô.

Durante deslocamentos lineares, a correção angular é combinada com a velocidade base definida para o movimento. Dessa forma, as velocidades de referência dos lados direito e esquerdo podem ser expressas por:

$$\omega_{\text{ref},R}(t) = \omega_{\text{base}}(t) + u_\theta(t) \quad (4.13)$$

$$\omega_{\text{ref},L}(t) = \omega_{\text{base}}(t) - u_\theta(t) \quad (4.14)$$

em que $\omega_{\text{ref},R}(t)$ e $\omega_{\text{ref},L}(t)$ representam as velocidades angulares de referência para os lados direito e esquerdo, respectivamente, e $\omega_{\text{base}}(t)$ representa a velocidade angular base do movimento. Assim, caso o robô apresente desvio de orientação, a velocidade de um lado é aumentada enquanto a do outro é reduzida, produzindo uma correção diferencial da trajetória.

Durante curvas ou manobras de rotação sobre o próprio eixo, as referências de velocidade são definidas com sinais opostos para os dois lados do robô:

$$\omega_{\text{ref},R}(t) = +u_\theta(t) \quad (4.15)$$

$$\omega_{\text{ref},L}(t) = -u_\theta(t) \quad (4.16)$$

Essa configuração permite que os lados esquerdo e direito girem em sentidos opostos, produzindo a rotação do robô em torno do próprio eixo. Na versão atual do sistema, a variação dos encoders é utilizada como critério principal de parada da rotação, com uma constante de calibração denominada `TURN_ENCODER_SCALE`. Essa constante foi introduzida para ajustar a relação entre a rotação estimada pelos encoders e a rotação efetivamente observada no robô durante os testes práticos.

A adoção dos encoders como referência principal para controle de rotação ocorreu após ensaios indicarem que o magnetômetro QMC5883L sofre interferências significativas durante o acionamento dos motores. Por esse motivo, o QMC5883L foi mantido como referência auxiliar de orientação, enquanto o controle direto das rotações passou a ser baseado majoritariamente na odometria dos encoders e nos critérios de validação implementados no sistema.

Dessa forma, a malha de direção atua em nível superior, definindo a correção angular necessária para manter ou alterar a orientação do robô, enquanto a malha de velocidade atua em nível inferior, ajustando o PWM dos motores para que as velocidades de referência sejam alcançadas. Essa organização em duas malhas facilita a calibração do sistema, permite compensar diferenças entre os motores e melhora a previsibilidade da movimentação do robô durante a navegação em grid.

4.3.3 Sensoriamento e Mapeamento do Ambiente

O sensor de distância VL53L0X foi incorporado ao sistema com o objetivo inicial de fornecer medições frontais de distância durante o deslocamento do robô, de modo que essas informações pudessem ser utilizadas tanto para a identificação de obstáculos quanto para a atualização de uma matriz de ocupação (*grid map*). Nesse modelo, o ambiente é representado de forma discretizada, e cada célula da matriz pode assumir estados como livre, ocupada ou desconhecida, conforme as informações obtidas durante a navegação.

Entretanto, ao longo do desenvolvimento do protótipo, a estratégia de mapeamento completo em tempo real precisou ser simplificada devido a limitações práticas encontradas durante a integração mecânica e eletrônica do sistema. Assim, na versão implementada e validada experimentalmente, o robô passou a operar a partir de um mapa previamente fornecido, enquanto o sensor VL53L0X foi utilizado principalmente como mecanismo de detecção de obstáculos à frente do robô. Dessa forma, o sensor não foi utilizado para construir integralmente o mapa do ambiente durante a execução, mas sim como uma camada reativa de segurança associada à navegação em grid.

Essa decisão permitiu manter a lógica de planejamento de rotas baseada no mapa discreto, ao mesmo tempo em que o sistema continuou capaz de reagir a obstáculos não previstos na representação inicial. Durante a movimentação entre células, as leituras do VL53L0X são avaliadas para verificar se há algum objeto no caminho imediato do robô. Caso uma distância inferior ao limite definido seja detectada, o sistema interrompe ou aborta a movimentação, evitando a colisão e registrando a ocorrência como obstáculo identificado durante a navegação.

Durante a etapa de prototipagem, o sensor VL53L0X permaneceu em uma condição de montagem provisória, em razão das limitações mecânicas da estrutura utilizada e da fragilidade das conexões por jumpers. A ausência de um suporte rígido e alinhado à região frontal do robô dificultou o uso confiável do sensor para atualização contínua do mapa, pois pequenas variações de posicionamento poderiam alterar a direção do feixe de medição e comprometer a interpretação espacial das leituras. Por esse motivo, a validação concentrou-se na comunicação com o ESP32, na obtenção das medições de distância e na integração dessas informações à lógica de detecção de obstáculos.

Portanto, no escopo final deste trabalho, o VL53L0X atua como sensor de proximidade

frontal para detecção de obstáculos, e não como sensor principal de mapeamento autônomo do ambiente. O mapeamento em grid permanece como estrutura de representação espacial e planejamento de rotas, porém o mapa utilizado nos testes é fornecido previamente ao sistema. Essa distinção é importante, pois separa a função de planejamento global, baseada no mapa conhecido, da função de reação local, baseada nas leituras do sensor de distância.

Em uma versão futura do protótipo, recomenda-se reposicionar e fixar o VL53L0X na região frontal do robô com suporte mecânico adequado, garantindo alinhamento estável com a direção de deslocamento. Com uma montagem mais rígida e calibrada, as medições do sensor poderiam ser utilizadas não apenas para detecção de obstáculos, mas também para atualização dinâmica do mapa de ocupação, aproximando o sistema da proposta inicial de automapeamento do ambiente.

4.3.4 Planejamento de Rota com Algoritmo A*

Com base na matriz de ocupação gerada, o planejamento de caminho entre a posição atual e o destino desejado é realizado por meio do algoritmo A* (*A-star*). Esse algoritmo é amplamente utilizado em sistemas robóticos por combinar eficiência e precisão, realizando uma busca heurística que minimiza o custo total do caminho considerando a distância percorrida e a distância estimada até o objetivo.

O resultado do algoritmo é uma sequência de coordenadas representando a trajetória ideal, a qual é traduzida em comandos de movimento (frente, curva à direita, curva à esquerda, etc.) e enviada ao sistema de controle de baixo nível.

A Figura 12 ilustra um exemplo visual da aplicação do algoritmo A* sobre uma matriz de ocupação, destacando um trajeto seguro entre o ponto de origem e o destino. Embora essa figura não represente o algoritmo efetivamente utilizado pelo robô, ela ajuda a visualizar o princípio de planejamento de caminhos adotado neste trabalho.

7	6	5	6	7	8	9	10	11		19	20	21	22
6	5	4	5	6	7	8	9	10		18	19	20	21
5	4	3	4	5	6	7	8	9		17	18	19	20
4	3	2	3	4	5	6	7	8		16	17	18	19
3	2	1	2	3	4	5	6	7		15	16	17	18
2	1	0	1	2	3	4	5	6		14	15	16	17
3	2	1	2	3	4	5	6	7		13	14	15	16
4	3	2	3	4	5	6	7	8		12	13	14	15
5	4	3	4	5	6	7	8	9	10	11	12	13	14
6	5	4	5	6	7	8	9	10	11	12	13	14	15

Figura 12 – Exemplo de caminho traçado pelo algoritmo A* sobre um mapa de ocupação.

Fonte: [@nicholas.w.swift at Medium.com](#)

Visando a economia de bateria e a eficiência operacional, o sistema de navegação é projetado para seguir a rota planejada de forma otimizada, evitando manobras desnecessárias e reduzindo o tempo total de deslocamento. Nesse sentido, as curvas são penalizadas ao máximo, pois exigem maior esforço ao alterar o sentido de rotação dos motores, consomem mais energia e tendem a tornar a trajetória mais lenta.

4.3.5 Execução da Navegação

Após a definição da rota pelo algoritmo de planejamento, o caminho obtido é convertido em uma sequência de comandos elementares de movimento. Como o ambiente é representado por uma matriz discreta, cada etapa da rota corresponde à transição entre uma célula atual e uma célula vizinha. A partir da diferença entre as coordenadas consecutivas do caminho, o sistema determina se o robô deve manter sua orientação atual, realizar uma rotação ou avançar para a próxima célula.

Na implementação desenvolvida, os principais comandos de movimentação considerados foram:

- **Avançar uma célula:** desloca o robô para frente por aproximadamente uma célula do grid, correspondente a 30 cm. Esse comando é utilizado quando a próxima coordenada da rota está à frente da orientação atual do robô.
- **Virar à esquerda:** executa uma rotação do robô para o lado esquerdo, geralmente associada a uma mudança de orientação de aproximadamente 90°, dependendo da direção exigida pela próxima célula do caminho.
- **Virar à direita:** executa uma rotação do robô para o lado direito, também utilizada para alinhar o robô com a próxima célula da rota planejada.
- **Rotacionar para orientação alvo:** ajusta a pose angular do robô para uma orientação desejada antes de iniciar o deslocamento linear. Esse comando é utilizado quando a diferença entre a orientação atual e a direção da próxima célula exige uma correção angular.
- **Parar:** interrompe imediatamente o acionamento dos motores. Esse comando é utilizado ao final de cada deslocamento, ao término de uma rotação, em caso de obstáculo detectado ou quando ocorre o cancelamento da navegação.
- **Abortar navegação:** interrompe a execução da rota atual e impede a continuidade dos comandos planejados. Esse comportamento é acionado quando o sensor frontal identifica um obstáculo inesperado ou quando alguma condição de segurança impede a continuidade do movimento.

Durante a execução da navegação, o robô não realiza o percurso de forma contínua e irrestrita. Em vez disso, a trajetória é executada de maneira incremental: o robô se orienta em

direção à próxima célula, avança aproximadamente 30 cm, interrompe o movimento, atualiza sua pose estimada e então prossegue para a próxima etapa da rota. Essa estratégia facilita a correção de orientação entre células e reduz o acúmulo de erro durante a navegação.

O controle dos deslocamentos lineares é realizado com base na odometria dos encoders, enquanto as rotações utilizam a variação estimada pelos encoders como critério principal de parada. Para compensar diferenças entre a rotação calculada e a rotação observada no protótipo, foi utilizada uma constante de calibração denominada `TURN_ENCODER_SCALE`. O magnetômetro QMC5883L permanece como referência auxiliar de orientação, mas não é utilizado como critério principal de parada das rotações devido às interferências observadas durante o acionamento dos motores.

Durante todo o trajeto, o sensor VL53L0X monitora a presença de obstáculos imprevistos à frente do robô. Caso um obstáculo seja detectado durante o deslocamento para a próxima célula, o sistema interrompe a locomoção e aborta a execução atual. Na implementação testada, o mapa 7x7 utilizado nos experimentos é preservado, sem sobrescrever células dinamicamente, para evitar que leituras momentâneas do sensor modifiquem a representação base do ambiente durante a calibração.

Essa estratégia caracteriza a navegação como um processo híbrido: o planejamento global define a sequência de células a ser percorrida, enquanto o controle local executa comandos de movimento e reage a obstáculos detectados durante o trajeto. Dessa forma, o robô é capaz de executar tarefas simples de deslocamento autônomo, seguindo uma rota planejada, monitorando o ambiente imediato e interrompendo sua movimentação quando uma condição de risco é identificada.

4.3.6 Calibração de Giro e Rejeição de Leituras Instáveis

Durante os testes de rotação, verificou-se que o uso exclusivo do magnetômetro para finalizar curvas não era suficientemente confiável, pois as leituras do QMC5883L apresentavam saltos significativos quando os motores e a ponte H estavam acionados. Para contornar esse problema, a lógica de giro passou a utilizar os encoders como critério principal de parada, com uma constante de calibração denominada `TURN_ENCODER_SCALE`. Essa constante permite ajustar a relação entre os pulsos medidos pelos encoders e o ângulo físico efetivamente executado pelo robô.

Além disso, foi incorporada uma lógica de rejeição de leituras anômalas. No caso dos encoders, saltos incompatíveis com a velocidade esperada são descartados para evitar que um pulso espúrio encerre uma curva prematuramente. No caso do QMC5883L, leituras com inovação angular excessiva são ignoradas pelo estimador de posição, impedindo que interferências magnéticas atualizem de forma indevida a orientação do robô.

4.4 Interface Web e Sistema de Arquivos Flash

A interface web desenvolvida para o sistema permite o monitoramento e controle remoto do robô por meio de um navegador padrão, sem a necessidade de softwares adicionais. Esta interface é hospedada diretamente no microcontrolador ESP32, garantindo autonomia e independência de conexões externas. Os arquivos estáticos, compostos por páginas HTML, folhas de estilo CSS e scripts JavaScript, são armazenados e carregados utilizando o sistema de arquivos SPIFFS, otimizado para dispositivos embarcados com memória limitada.

Para viabilizar a comunicação bidirecional em tempo real entre a interface e o robô, emprega-se o protocolo WebSocket. Esse protocolo possibilita a atualização contínua do status dos sensores, bem como o envio imediato de comandos de movimentação e controle, garantindo uma resposta rápida e eficiente. O uso de WebSocket também minimiza a sobrecarga de comunicação e otimiza o consumo de recursos do ESP32, assegurando estabilidade e desempenho adequados para a aplicação.

4.5 Integração e Montagem Mecânica do Robô

O chassi do robô foi projetado e montado utilizando materiais leves e resistentes, priorizando a eficiência energética e a mobilidade. As rodas foram acopladas a motores de corrente contínua equipados com encoders rotativos, permitindo o controle preciso da velocidade e do deslocamento, fundamentais para a implementação de algoritmos de navegação autônoma.

Os sensores foram estrategicamente posicionados para melhorar a confiabilidade das leituras. O sensor ToF foi colocado na região frontal do chassi para detecção de obstáculos no sentido de deslocamento, enquanto o magnetômetro QMC5883L deve permanecer o mais afastado possível da ponte H, dos motores, da bateria e dos cabos de corrente elevada. A fiação e os cabos foram organizados de forma cuidadosa, utilizando fixadores, para facilitar a manutenção, reduzir interferências eletromagnéticas e evitar danos mecânicos durante o funcionamento.

A alimentação do sistema é fornecida por uma bateria de íon-lítio (Li-ion) 2S, escolhida por sua densidade energética e capacidade de recarga. A distribuição de energia foi organizada com conector XT60, chave SS12F46, regulador LM2596 e módulo L298N. O uso de uma PCB carrier é previsto para reduzir falhas de contato, facilitar a fixação dos módulos e tornar a manutenção elétrica mais segura.

4.6 Descrição dos Testes

Para avaliar o funcionamento do sistema desenvolvido, foram definidos testes experimentais com o objetivo de verificar o comportamento do robô em tarefas básicas de navegação em grid, deslocamento linear, rotação, localização estimada e detecção de obstáculos. Os testes foram planejados

considerando o mapa discreto utilizado no projeto, composto por células de $30\text{ cm} \times 30\text{ cm}$, no qual a posição do robô é representada por coordenadas (x, y) .

Em todos os testes, considerou-se que o robô inicia sua operação em uma célula conhecida do grid, com orientação inicial definida. A execução foi acompanhada por meio da interface web embarcada, dos registros de telemetria e dos logs gerados pelo sistema. As principais métricas utilizadas para avaliação foram: conclusão ou não da rota planejada, distância estimada percorrida por célula, erro entre posição final esperada e posição final estimada, comportamento das rotações, detecção de obstáculos e ocorrência de abortos de navegação.

Teste 1 - Deslocamento linear entre células

O primeiro teste teve como objetivo avaliar a capacidade do robô de executar deslocamentos lineares entre células consecutivas do grid. Nesse ensaio, o robô parte de uma posição inicial conhecida e deve avançar por uma sequência de células livres, mantendo a orientação e interrompendo o movimento ao final de cada célula.

Para esse teste, definiu-se como posição inicial a coordenada $(1, 1)$ e como posição final esperada a coordenada $(1, 5)$, considerando deslocamentos sucessivos no eixo vertical do mapa. Dessa forma, o robô deve executar quatro deslocamentos lineares consecutivos, cada um correspondente a uma célula de aproximadamente 30 cm.

As métricas observadas nesse teste foram:

- **Número de células concluídas:** quantidade de células percorridas com sucesso até a posição final ou até uma interrupção da navegação;
- **Distância estimada por célula:** valor calculado pela odometria ao final de cada deslocamento, comparado com a referência de 30 cm;
- **Erro de deslocamento por célula:** diferença entre a distância estimada e a distância nominal da célula;
- **Manutenção da orientação:** avaliação do desvio angular acumulado durante o deslocamento linear;
- **Ocorrência de obstáculos detectados:** identificação de leituras do sensor VL53L0X que interrompam a execução do movimento.

Esse teste permite verificar se o sistema de odometria por encoders é capaz de estimar deslocamentos compatíveis com o tamanho das células do grid, além de avaliar se a lógica de navegação incremental consegue avançar célula por célula sem executar o percurso de forma contínua e descontrolada.

Teste 2 - Navegação com mudança de direção

O segundo teste teve como objetivo avaliar a execução de uma rota que exige mudança de orientação durante o percurso. Diferentemente do teste anterior, neste ensaio o robô deve combinar deslocamentos lineares e comandos de rotação, verificando a integração entre o planejamento de rota, o controle de direção e o controle de velocidade dos motores.

Para esse teste, definiu-se como posição inicial a coordenada (1, 1) e como posição final esperada uma célula não alinhada diretamente com a orientação inicial do robô, por exemplo (3, 3). Nesse cenário, a rota planejada exige que o robô avance por células livres, realize ao menos uma rotação para a esquerda ou para a direita e continue o deslocamento até a posição final.

As métricas observadas nesse teste foram:

- **Conclusão da rota planejada:** verificação se o robô atingiu a coordenada final prevista pelo planejador;
- **Quantidade de rotações executadas:** número de comandos de giro realizados durante a rota;
- **Erro angular após rotação:** diferença entre o ângulo alvo e o ângulo estimado após a manobra;
- **Tempo aproximado de execução da rota:** intervalo entre o início da navegação e a conclusão ou interrupção do percurso;
- **Estabilidade do controle:** observação de travamentos, oscilações, correções excessivas ou abortos de navegação durante a execução;
- **Comportamento da constante `TURN_ENCODER_SCALE`:** verificação se a calibração aplicada à rotação por encoders produz giros compatíveis com os ângulos esperados.

Esse teste permite avaliar o funcionamento conjunto das malhas de controle de velocidade e direção, bem como a capacidade do robô de converter uma rota planejada em comandos elementares de movimento, como avançar uma célula, virar à esquerda, virar à direita e parar ao final de cada etapa.

Teste 3 - Detecção de obstáculo durante a navegação

Além dos testes de deslocamento e rotação, foi definido um teste específico para avaliar a reação do sistema diante da presença de um obstáculo inesperado. Nesse ensaio, o robô inicia a navegação a partir de uma posição conhecida e recebe um destino que exige deslocamento linear por

células livres. Durante a execução, um obstáculo é posicionado à frente do robô, dentro da faixa de detecção do sensor VL53L0X.

Para esse teste, pode-se definir como posição inicial a coordenada (1, 1) e como posição final esperada a coordenada (1, 5). O obstáculo deve ser colocado em uma célula intermediária da trajetória, por exemplo próximo à coordenada (1, 4), de modo que o robô detecte a obstrução antes de completar o percurso.

As métricas observadas nesse teste foram:

- **Detecção do obstáculo:** verificação se o sensor VL53L0X identificou a presença do objeto à frente do robô;
- **Interrupção segura do movimento:** análise se o robô interrompeu os motores após a detecção;
- **Célula em que ocorreu a interrupção:** registro da posição estimada do robô no momento do abortamento;
- **Preservação do mapa base:** verificação se o mapa 7x7 permaneceu inalterado, sem sobrecrever dinamicamente células durante o teste;
- **Registro do evento em log:** confirmação de que a interface ou o sistema registrou a ocorrência de obstáculo detectado e navegação abortada.

Esse teste permite avaliar a função reativa do sensor VL53L0X no sistema final. Embora o objetivo inicial do projeto envolvesse o uso do sensor para mapeamento dinâmico, na implementação validada experimentalmente ele foi empregado principalmente como mecanismo de detecção de obstáculos, atuando como uma camada de segurança durante a navegação.

Teste 4 - Navegação em grade com A* modificado

O teste de navegação em grade teve como objetivo avaliar o funcionamento integrado do sistema de planejamento de rotas e execução de movimentos no mapa discreto utilizado pelo robô. Nesse ensaio, o robô parte de uma posição inicial conhecida, localizada no canto superior esquerdo da área navegável do mapa, e deve atingir uma posição final definida pelo usuário por meio da interface web.

Para esse teste, foi utilizado o mapa discreto 7x7 apresentado anteriormente, com bordas ocupadas e células internas livres ou bloqueadas conforme a configuração experimental. A rota entre a origem e o destino é calculada por uma versão modificada do algoritmo A*. Além do custo associado ao deslocamento entre células vizinhas, o planejador aplica uma penalização adicional para mudanças de direção, reduzindo a tendência de gerar trajetórias com muitas curvas consecutivas.

Essa modificação foi adotada porque, no protótipo físico, cada mudança de direção exige uma manobra de rotação, seguida de realinhamento e deslocamento linear. Assim, trajetórias com menor número de curvas tendem a ser mais adequadas para o robô, pois reduzem o acúmulo de erro angular, diminuem o tempo de execução e simplificam o controle dos motores.

Durante a execução, o caminho planejado pelo A* modificado é convertido em uma sequência de comandos elementares, como avançar uma célula, virar à esquerda, virar à direita, rotacionar para uma orientação alvo e parar. A cada etapa, o robô atualiza sua posição estimada, verifica a conclusão do movimento e monitora a presença de obstáculos por meio do sensor VL53L0X.

As métricas observadas nesse teste foram:

- **Caminho gerado pelo A*:** sequência de células calculada entre a posição inicial e a posição final;
- **Quantidade de células percorridas:** número de transições executadas ao longo do caminho planejado;
- **Quantidade de curvas planejadas:** número de mudanças de direção presentes na rota gerada;
- **Conclusão da rota:** verificação se o robô atingiu a coordenada final definida;
- **Erro de posição final:** diferença entre a posição final esperada e a posição estimada ao término da navegação;
- **Ocorrência de abortos:** verificação de interrupções causadas por obstáculos, erro de controle ou falha de execução;
- **Tempo de execução:** intervalo aproximado entre o início da navegação e a conclusão ou interrupção da rota.

Esse teste é o mais abrangente da validação experimental, pois combina o planejamento global de rotas com a execução local dos movimentos do robô. Além disso, permite verificar se a penalização de curvas no A* produz trajetórias mais compatíveis com as limitações físicas do protótipo e com a estratégia de deslocamento célula a célula adotada no sistema.

Síntese das métricas de avaliação

A Tabela 2 resume os principais testes definidos e as métricas utilizadas para avaliar o funcionamento do robô.

Os resultados obtidos a partir desses testes são apresentados e discutidos no capítulo seguinte, permitindo verificar se o robô foi capaz de executar deslocamentos compatíveis com o

Tabela 2 – Testes definidos para avaliação experimental do sistema

Teste	Objetivo	Métricas avaliadas
Deslocamento linear entre células	Verificar avanço em células consecutivas do grid	Número de células concluídas, distância estimada por célula, erro de deslocamento e manutenção da orientação
Navegação com mudança de direção	Avaliar integração entre deslocamento linear, rotação e planejamento de rota	Conclusão da rota, quantidade de rotações, erro angular, tempo de execução e comportamento do TURN_ENCODER_SCALE
Detecção de obstáculo	Verificar reação local diante de obstáculo inesperado	Detecção pelo VL53L0X, interrupção dos motores, célula de abortamento, preservação do mapa e registro em log
Navegação em grade com A* modificado	Avaliar o planejamento global e a execução completa de uma rota em grid	Caminho gerado, quantidade de células, número de curvas, conclusão da rota, erro de posição final, abortos e tempo de execução

tamanho das células, realizar rotações controladas, reagir a obstáculos e manter uma estimativa coerente de sua posição durante a navegação.

5 Resultados

Este capítulo apresenta os principais resultados obtidos durante o desenvolvimento e a validação experimental do robô móvel autônomo. A análise foi organizada em três resultados principais: a montagem final do protótipo físico, a interface web desenvolvida para operação e monitoramento do sistema, e o funcionamento integrado do robô durante os testes de navegação.

O primeiro resultado corresponde à consolidação do protótipo físico, incluindo a integração entre chassi, motores, encoders, sensores, ponte H, módulo de alimentação e ESP32. Nessa etapa, são apresentadas as características construtivas do robô, suas dimensões físicas e a disposição geral dos componentes embarcados.

O segundo resultado refere-se à interface web final, utilizada para acompanhar o estado do sistema e enviar comandos ao robô. A interface reúne recursos como visualização do mapa em grade, indicação da posição estimada, comandos manuais de movimentação, seleção de destino e monitoramento de dados de telemetria. Essa parte também permite relacionar o tamanho físico do robô com as células do mapa, definidas como quadrados de 30 cm $\times$ 30 cm.

O terceiro resultado trata do funcionamento integrado do sistema, utilizando os testes definidos no capítulo de Materiais e Métodos. Nessa etapa, são avaliados o deslocamento entre células, as rotações, a detecção de obstáculos e a navegação em grade com rotas geradas pelo algoritmo A* modificado, que penaliza mudanças de direção.

Para a análise deste resultado, foram utilizados os testes definidos na Seção 4.6 do Capítulo 4, especialmente os Testes 4.6, 4.6, 4.6 e 4.6.

O objetivo deste capítulo não é apenas relatar que o protótipo foi montado, mas demonstrar quais partes do sistema apresentaram comportamento funcional, quais limitações foram observadas e quais ajustes técnicos foram necessários para tornar a navegação mais previsível. Por esse motivo, os resultados são apresentados juntamente com uma discussão técnica dos principais problemas identificados durante os ensaios.

5.1 Mapa de Teste e Planejamento de Rotas

Para os ensaios de navegação foi utilizado um mapa discreto 7x7, no qual as bordas foram marcadas como células ocupadas e a região interna como área navegável. A célula inicial do robô foi definida em (1, 1), considerando a origem (0, 0) no canto superior esquerdo. Essa escolha simplifica a representação espacial e permite que os deslocamentos sejam tratados como transições discretas entre células adjacentes.

A Figura 13 apresenta o mapa utilizado nos testes de navegação por células. Nessa re-

apresentação, as células escuras indicam regiões ocupadas ou intransponíveis, enquanto as células claras representam posições livres para navegação. O robô inicia sua movimentação na célula (1, 1), indicada visualmente no mapa, e os destinos dos testes são definidos dentro da área livre do grid.

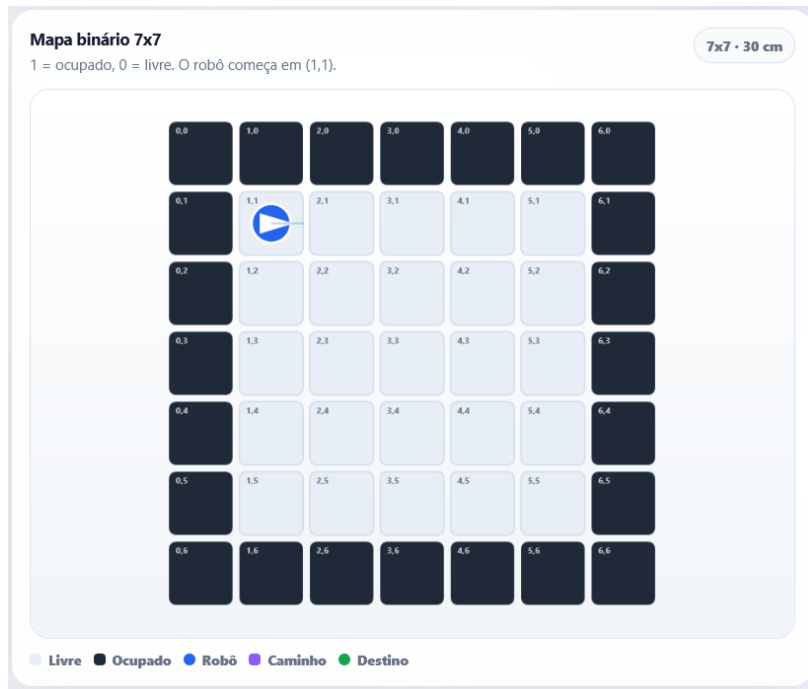


Figura 13 – Mapa 7x7 utilizado nos testes de navegação por células.

O planejamento de rotas foi realizado por meio de uma versão modificada do algoritmo A*. O algoritmo calcula uma sequência de células entre a posição inicial e o destino definido pelo usuário, considerando as células livres e ocupadas do mapa. Além do custo de deslocamento entre células adjacentes, foi adicionada uma penalização para mudanças de direção, com o objetivo de reduzir a quantidade de curvas geradas na rota final.

Essa modificação é importante para o protótipo físico, pois cada curva exige uma manobra de rotação, seguida de realinhamento e deslocamento linear. Assim, mesmo quando existem caminhos com o mesmo comprimento em número de células, a versão modificada do A* tende a favorecer trajetórias com menor quantidade de mudanças de direção, tornando a execução mais compatível com as limitações mecânicas do robô.

De forma simplificada, o custo de uma transição entre duas células pode ser representado por:

$$C = C_{\text{deslocamento}} + C_{\text{curva}} \quad (5.1)$$

em que $C_{\text{deslocamento}}$ representa o custo de avançar para uma célula vizinha e C_{curva} representa uma penalização adicional aplicada quando a direção do movimento muda em relação ao

trecho anterior. Dessa forma, o planejador não apenas busca um caminho válido até o destino, mas também procura reduzir manobras desnecessárias.

Após a geração da rota, o navegador em grade converte a sequência de células em comandos elementares de movimento, como avançar uma célula, virar à esquerda, virar à direita, rotacionar para uma orientação alvo e parar. Esse processo permite relacionar diretamente o resultado do planejamento com a execução física do robô.

5.2 Resultados do Deslocamento Linear

O teste de deslocamento linear teve como objetivo verificar se o robô era capaz de avançar entre células consecutivas do grid utilizando a odometria dos encoders como referência de distância. Para esse ensaio, foi definida uma rota simples em linha reta, com posição inicial em (1, 1) e posição final em (1, 5). Considerando que cada célula possui 30 cm, essa rota exige quatro deslocamentos lineares consecutivos, totalizando aproximadamente 1,20 m.

A rota planejada para esse teste pode ser representada pela sequência:

$$(1, 1) \rightarrow (1, 2) \rightarrow (1, 3) \rightarrow (1, 4) \rightarrow (1, 5) \quad (5.2)$$

Essa sequência não exige curvas, pois todas as células pertencem ao mesmo alinhamento no grid. Portanto, o robô deve apenas manter sua orientação e executar o comando de avanço de uma célula quatro vezes consecutivas. Esse teste foi utilizado para avaliar a consistência da medição de distância por encoder antes da análise de rotas mais complexas envolvendo mudanças de direção.

Os testes de deslocamento linear indicaram que o controle por encoders apresentou comportamento consistente para células de aproximadamente 30 cm. Em execuções registradas, o robô concluiu deslocamentos individuais com valores próximos ao comprimento esperado, frequentemente entre 0,286 m e 0,296 m. A Tabela 3 apresenta exemplos extraídos dos registros de navegação.

Tabela 3 – Exemplos de deslocamentos lineares registrados em navegação por células.

Registro	Célula/trecho	Distância registrada (m)	Erro em relação a 0,30 m
16/06 19:15	célula 1/4	0,286	-4,67%
16/06 19:15	célula 2/4	0,291	-3,00%
16/06 19:15	célula 3/4	0,286	-4,67%
16/06 19:15	célula 4/4	0,286	-4,67%
16/06 20:09	célula 1/4	0,286	-4,67%
16/06 20:09	célula 2/4	0,286	-4,67%
16/06 20:09	célula 3/4	0,286	-4,67%

A Tabela 3 mostra que os deslocamentos medidos ficaram próximos da distância nominal de 0,30 m por célula. Os erros observados nos exemplos apresentados variaram aproximadamente entre 3% e 5%, indicando que a odometria por encoders foi capaz de estimar deslocamentos compatíveis com o tamanho definido para as células do mapa.

Esses desvios podem ser atribuídos a fatores como atrito das rodas, irregularidade do piso, inércia do conjunto, resolução dos encoders, tolerâncias mecânicas da montagem e diferença de desempenho entre os motores. Ainda assim, o resultado é suficiente para sustentar a navegação por células, pois a execução da rota em grid depende diretamente da capacidade de avançar distâncias próximas ao tamanho definido para cada célula.

5.3 Resultados do Controle de Rotação

A rotação do robô foi a etapa que apresentou maior complexidade experimental. Inicialmente, foi avaliada a possibilidade de utilizar o magnetômetro QMC5883L como referência principal para encerramento das curvas. Entretanto, os testes demonstraram que o uso isolado do QMC5883L durante o acionamento dos motores não é confiável, pois as leituras de orientação apresentaram saltos abruptos durante as curvas.

Em registros de teste com giro baseado somente no QMC5883L, o heading variou de forma incompatível com o movimento físico observado. Em um dos ensaios, a leitura passou rapidamente de valores próximos a 18° para aproximadamente 99° , fazendo com que o sistema interpretasse falsamente que o alvo angular havia sido atingido. Em outro conjunto de registros, foram observadas inversões sucessivas de sinal e variações próximas de 180° durante a tentativa de curva. Esse comportamento indica interferência eletromagnética causada por motores, cabos, bateria e ponte H, além da sensibilidade natural de magnetômetros quando instalados próximos a correntes elevadas.

A Tabela 4 resume a evolução das estratégias de controle de rotação testadas. Ela mostra a transição entre o uso do QMC5883L como referência principal, o uso inicial da odometria dos encoders e a adoção posterior de uma constante de calibração para o controle angular.

Tabela 4 – Evolução das estratégias de controle de rotação testadas.

Estratégia testada	Resultado observado	Decisão adotada
Giro com QMC5883L como referência única	Leituras de heading instáveis, com saltos abruptos e encerramento falso de curvas	Rejeitada como estratégia principal de parada
Giro por encoder sem calibração final	O sistema estimava que havia completado 90°, mas o giro físico observado era incompatível	Necessidade de ajuste de escala angular
Giro por encoder com <code>TURN_ENCODER_SCALE</code> definido	Estimativa angular interna mais próxima do alvo	Mantida como estratégia principal de controle de rotação

Com base nesses resultados, a estratégia de controle foi alterada: os encoders passaram a ser a referência principal para controle do giro, enquanto o QMC5883L passou a ser tratado como referência auxiliar, usada apenas quando sua leitura se mostra estável. Essa decisão reduziu a dependência do sistema em relação a um sensor suscetível a interferências magnéticas durante o acionamento dos motores.

A Tabela 5 apresenta um exemplo de calibração angular com a constante `TURN_ENCODER_SCALE`. No exemplo registrado, o valor bruto estimado pelos encoders foi de 179,68°. Após a aplicação da escala 0,50, o ângulo estimado passou a 89,84°, valor muito próximo do alvo de 90°.

Tabela 5 – Exemplo de calibração angular com `TURN_ENCODER_SCALE`.

Alvo	Raw encoder	Escala	Ângulo escalado	Erro interno
90,00°	179,68°	0,50	89,84°	0,16°

Esse resultado indica que, do ponto de vista da odometria interna, a calibração reduziu significativamente o erro de escala angular. Entretanto, a Tabela 5 ainda não substitui uma medição física externa do ângulo real executado pelo robô. Por isso, recomenda-se complementar essa análise com medições realizadas por marcação no piso, transferidor, câmera ou análise quadro a quadro de vídeo.

Apesar da melhora obtida com a escala angular, os registros também evidenciaram que a fase fina do giro ainda necessita refinamento. Em alguns testes, o robô permanecia por vários ciclos com erro residual de aproximadamente 10,61° ou 2,25°, enquanto a rotação medida pelos encoders não avançava. Esse comportamento sugere que, nas velocidades finais mais baixas, o conjunto motor, ponte H e rodas pode não gerar torque suficiente para vencer o atrito estático.

Portanto, a solução mais adequada não é alterar novamente a escala angular, mas ajustar a lógica de fase fina, elevando a rotação mínima ou adicionando pulsos curtos de destravamento quando o encoder permanece sem variação.

5.4 Interface Web e Monitoramento

A interface web embarcada permitiu o envio de comandos de navegação e o acompanhamento visual do estado do robô. A Figura 14 apresenta a interface principal de controle utilizada durante os testes. Nessa tela, é possível visualizar o mapa em grade, acompanhar a posição estimada do robô, observar sua orientação por meio da seta direcional, acessar os comandos de movimentação, além da possibilidade de definir o destino que o algoritmo de navegação A* irá seguir.

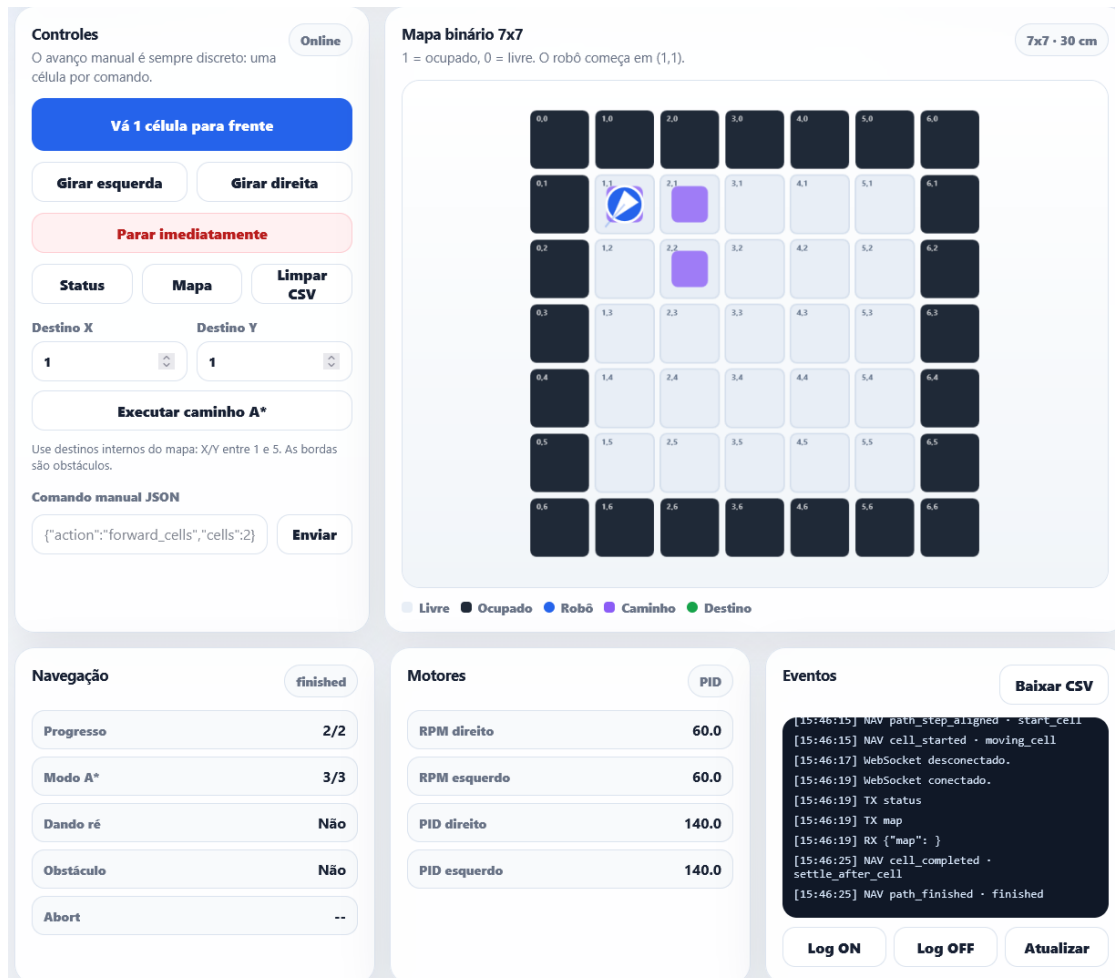


Figura 14 – Interface principal de controle do robô durante os testes.

A interface apresentada na Figura 14 foi importante para a validação experimental porque permitiu comparar o comportamento físico do robô com a representação lógica exibida no mapa. Durante os testes, essa visualização ajudou a verificar se a célula estimada, a orientação e os comandos executados estavam coerentes com o comportamento esperado.

Além da interface principal, o sistema conta com uma tela de monitoramento e logs em tempo real. A Figura 15 apresenta essa interface, utilizada para acompanhar mensagens do sistema durante a execução de comandos, deslocamentos, rotações e eventos de navegação.

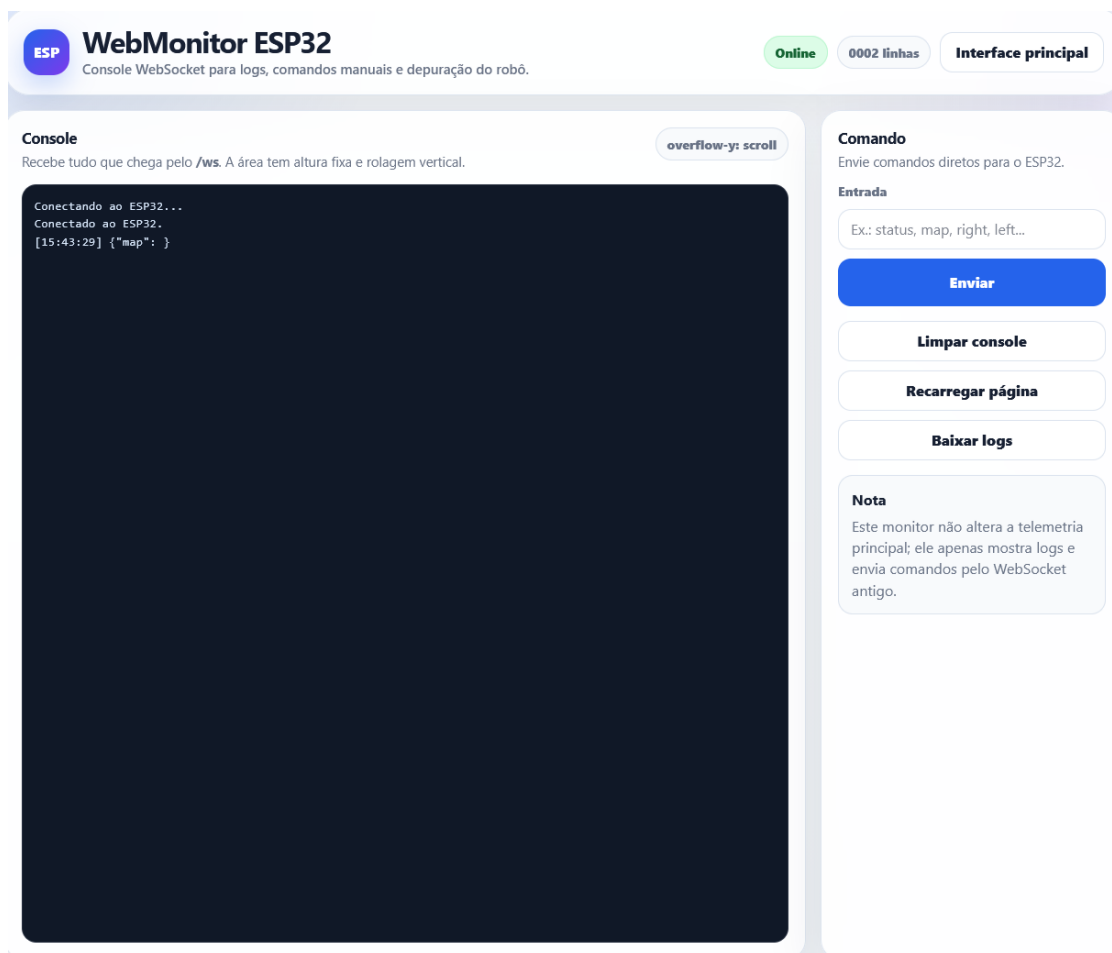


Figura 15 – Interface de monitoramento e logs em tempo real.

A interface de monitoramento mostrada na Figura 15 permitiu observar registros como início e fim de movimentos, conclusão de células, leituras de sensores, mensagens de obstáculo detectado, valores de rotação por encoder e informações de depuração. Esse recurso foi essencial para interpretar os resultados experimentais, pois muitos ajustes realizados no controle de movimento foram baseados nos registros exibidos durante os testes.

O monitor web também foi atualizado para apresentar aparência semelhante ao painel de controle e limitar a expansão do console de logs, utilizando rolagem vertical. Essa modificação foi necessária porque os testes geram grande volume de mensagens, principalmente durante curvas e navegação por múltiplas células. Com a limitação visual do console, a interface permanece utilizável mesmo durante execuções longas.

5.5 Síntese dos Resultados Obtidos

A síntese dos resultados deve destacar não apenas os módulos testados individualmente, mas também a validação do planejamento de rotas com o algoritmo A* modificado. Esse ponto é importante porque o planejamento em grade representa uma das principais funcionalidades do sistema:

gerar uma rota a partir de uma posição inicial e um destino definido, convertendo essa rota em comandos executáveis pelo robô.

Para demonstrar o funcionamento do A* modificado, foram considerados dois exemplos de destino a partir da mesma posição inicial (1, 1). O primeiro exemplo utiliza um destino alinhado com a posição inicial, enquanto o segundo utiliza um destino que exige mudança de direção. Essa comparação permite verificar tanto o cálculo de rotas simples quanto o comportamento do planejador em trajetórias com curvas.

No primeiro exemplo, foi definida a posição inicial (1, 1) e a posição final (5, 1). Nesse caso, a rota planejada pelo A* modificado é predominantemente linear:

$$R_1 = \{(1, 1), (2, 1), (3, 1), (4, 1), (5, 1)\} \quad (5.3)$$

Essa rota possui quatro transições entre células e não exige mudança de direção, sendo adequada para verificar se o planejador gera corretamente um caminho direto quando não há obstáculos intermediários.

No segundo exemplo, foi definida a mesma posição inicial (1, 1), porém com posição final (5, 5). Nesse caso, a rota exige deslocamento em dois eixos do mapa. Considerando a penalização de curvas, o A* modificado tende a favorecer uma trajetória com menor número de mudanças de direção, como:

$$R_2 = \{(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)\} \quad (5.4)$$

Essa segunda rota possui oito transições entre células e uma mudança principal de direção. O resultado é compatível com a proposta de penalizar curvas, pois o planejador evita trajetórias com alternância excessiva entre movimentos horizontais e verticais.

A Tabela 6 resume os dois exemplos de rota calculados pelo A* modificado, destacando a posição inicial, a posição final, o número de células percorridas e a quantidade de mudanças de direção.

Tabela 6 – Exemplos de rotas calculadas pelo A* modificado.

Exemplo	Início	Destino	Transições	Mudanças de direção
R_1	(1, 1)	(5, 1)	4	0
R_2	(1, 1)	(5, 5)	8	1

A Tabela 6 evidencia que o planejador é capaz de gerar rotas válidas entre pontos distintos do mapa. Além disso, o segundo exemplo mostra o efeito da penalização de curvas, pois a rota resultante concentra a mudança de direção em um único ponto, favorecendo uma execução mais simples para o robô físico.

A Tabela 7 sintetiza os principais resultados obtidos até o momento, considerando deslocamento linear, rotação, sensores, interface web e planejamento de rotas.

Tabela 7 – Síntese dos resultados experimentais do protótipo.

Módulo avaliado	Resultado obtido	Situação atual
Deslocamento linear	Células próximas de 0,30 m, com registros típicos entre 0,286 m e 0,296 m	Funcional, com necessidade de calibração fina
Rotação	Controle por encoder tornou-se mais confiável após uso de TURN_ENCODER_SCALE	Funcional em desenvolvimento; fase fina precisa ajuste
QMC5883L	Leituras instáveis durante motores ligados, com saltos de heading	Usado apenas como referência auxiliar filtrada
VL53L0X	Deteção de obstáculo interrompeu navegação de forma segura	Funcional para abortamento de rota
Mapa 7x7 e A* modificado	Rotas foram geradas a partir de pontos finais definidos, com penalização de curvas e conversão em comandos de movimento	Funcional em ambiente controlado; necessita registro visual/vídeo da execução física completa
Interface web	Permitiu controle, telemetria, visualização de mapa, posição estimada e logs	Funcional, com melhorias visuais implementadas

A Tabela 7 resume o estágio funcional atingido pelo protótipo. Os resultados demonstram que o sistema alcançou integração entre planejamento, controle de movimento, sensoramento e interface web. O deslocamento linear apresentou medições próximas ao tamanho das células do grid, o controle de rotação evoluiu com a adoção da escala por encoder, o VL53L0X atuou como camada de segurança para detecção de obstáculos e a interface web permitiu acompanhar a navegação em tempo real.

O ponto mais importante para a validação do sistema integrado é o funcionamento do A* modificado em conjunto com a execução física da rota. Por isso, os exemplos R_1 e R_2 devem ser acompanhados de capturas da interface e, preferencialmente, de um vídeo do robô seguindo uma rota planejada. Esse material permitirá demonstrar na defesa que o sistema não apenas executa comandos isolados, mas também gera um caminho no mapa e transforma esse caminho em uma sequência de movimentos do protótipo.

6 Conclusão

Este trabalho apresentou o desenvolvimento de um veículo robótico autônomo de pequeno porte baseado no microcontrolador ESP32, com foco na construção de uma plataforma de baixo custo, modular e adequada à navegação em ambientes internos. A proposta envolveu a integração de sensores, atuadores, algoritmos de controle, planejamento de rotas e uma interface web embarcada, buscando validar uma arquitetura funcional para deslocamento autônomo em um ambiente discretizado por células.

Ao longo do desenvolvimento, foi possível implementar uma arquitetura embarcada capaz de integrar motores DC, encoders ópticos, sensor de distância VL53L0X, magnetômetro QMC5883L, módulo Ponte H L298N e comunicação Wi-Fi por meio do ESP32. A utilização do PlatformIO e da estrutura modular do firmware favoreceu a organização do código e a separação das responsabilidades entre os diferentes módulos do sistema, como controle dos motores, leitura dos sensores, navegação, mapeamento, comunicação e interface com o usuário. Além disso, o uso de recursos de execução concorrente com FreeRTOS permitiu organizar rotinas paralelas de controle, telemetria e sensoriamento, reduzindo o risco de bloqueios durante a operação do robô.

A interface web embarcada também se mostrou um elemento importante para a operação e depuração do sistema. Por meio dela, foi possível enviar comandos, selecionar destinos no mapa, visualizar a posição estimada do robô, acompanhar a orientação e monitorar eventos registrados durante os testes. A adoção de WebSockets contribuiu para uma comunicação bidirecional em tempo real entre o navegador e o ESP32, permitindo maior responsividade durante a execução das rotinas de navegação.

Os resultados obtidos indicaram que o sistema foi capaz de executar deslocamentos lineares por células com desempenho compatível com o modelo adotado no projeto. Os testes com células de aproximadamente 30 cm demonstraram que o controle de avanço baseado em encoders apresentou comportamento consistente, permitindo ao robô percorrer trechos discretizados do mapa e fixar sua posição lógica após a conclusão de cada célula. Essa estratégia foi importante para reduzir o acúmulo de erro e manter a navegação coerente com a representação matricial do ambiente.

No controle de rotação, os testes evidenciaram a necessidade de calibração específica para a conversão entre a rotação estimada pelos encoders e o giro efetivo do robô. A introdução do fator `TURN_ENCODER_SCALE` permitiu ajustar a relação entre a leitura dos encoders e o ângulo executado, tornando o controle de curvas mais previsível. Além disso, a implementação de mecanismos de rejeição de leituras anômalas contribuiu para aumentar a robustez do sistema diante de variações abruptas ou inconsistências nas leituras de rotação.

A avaliação do magnetômetro QMC5883L mostrou que, embora o sensor seja útil como referência auxiliar de orientação em condições estáveis, suas leituras apresentaram elevada sensibilidade a interferências durante o acionamento dos motores. Foram observados saltos abruptos no *heading* magnético, indicando que o magnetômetro não deve ser utilizado como principal referência para controle angular durante o movimento. Essa constatação levou à adoção de uma estratégia mais robusta, na qual os encoders são priorizados para a estimativa de deslocamento e rotação, enquanto o QMC5883L permanece como uma referência complementar, utilizada apenas quando suas leituras apresentam estabilidade suficiente.

O sensor VL53L0X foi integrado ao sistema com a finalidade de contribuir para a detecção de obstáculos e para a segurança da navegação. Embora a montagem física provisória tenha limitado seu posicionamento ideal durante parte dos testes, sua integração funcional com o firmware permitiu validar a leitura de distância e a lógica de interrupção da navegação quando um obstáculo era identificado. Dessa forma, o sensor demonstrou potencial para atuar como camada de segurança complementar ao planejamento de rotas e à navegação baseada em grid.

A metodologia de navegação utilizando mapa de ocupação 7x7 e planejamento de rota com algoritmo A* mostrou-se adequada para validar o funcionamento do protótipo em ambiente controlado. A representação do espaço por células simplificou a modelagem do ambiente e permitiu converter o caminho planejado em comandos discretos de movimentação. Essa abordagem, embora limitada em relação à complexidade de ambientes reais, mostrou-se suficiente para os objetivos deste trabalho, principalmente por permitir avaliar a integração entre planejamento, controle de movimento, sensoramento e interface com o usuário.

Apesar dos resultados positivos, o projeto apresentou limitações relevantes. A montagem em protoboard e o uso de conectores jumper introduziram fragilidade mecânica e possibilidade de mau contato durante a movimentação. A interferência eletromagnética sobre o magnetômetro também se mostrou um fator significativo, reforçando a necessidade de melhor posicionamento físico dos sensores e de uma organização elétrica mais robusta. Além disso, a validação experimental foi realizada em ambiente controlado, com número limitado de repetições e com um mapa simplificado, o que restringe a generalização dos resultados para cenários mais complexos.

Ainda assim, o protótipo desenvolvido atingiu os principais objetivos propostos. Foi possível construir uma plataforma funcional, baseada em ESP32, capaz de integrar sensoramento, controle de motores, planejamento de rotas, navegação por células e interface web embarcada. O trabalho também permitiu identificar, por meio dos testes, decisões importantes de arquitetura, como a priorização dos encoders para controle de movimentação, o uso do magnetômetro apenas como referência auxiliar e a necessidade de melhorias mecânicas para aumentar a confiabilidade do sistema.

Dessa forma, conclui-se que o projeto apresenta uma base viável para o desenvolvimento de robôs móveis autônomos de pequeno porte em contextos acadêmicos e educacionais. A arqui-

tetura proposta é modular, expansível e compatível com futuras melhorias de hardware e software. Além de atender ao objetivo de construir um protótipo funcional, o trabalho contribuiu para consolidar conhecimentos relacionados a sistemas embarcados, robótica móvel, controle PID, odometria, sensoriamento, comunicação em tempo real e planejamento de trajetórias.

6.1 Próximos Passos

Como continuidade deste trabalho, recomenda-se, inicialmente, substituir a montagem em proto-board por uma placa de circuito impresso ou por uma placa de suporte mais robusta, com conectores adequados e melhor fixação mecânica dos módulos. Essa alteração tende a reduzir problemas de mau contato, interferência, vibração e desconexão durante os testes, aumentando a confiabilidade do protótipo.

Outro ponto importante consiste no reposicionamento definitivo do sensor VL53L0X na região frontal do robô, com suporte mecânico apropriado. Com essa fixação, será possível realizar testes mais precisos de detecção de obstáculos à frente do robô, avaliando a distância real de detecção, a repetibilidade das leituras e a resposta do sistema em diferentes condições de navegação. Essa melhoria também permitirá utilizar o sensor de forma mais efetiva na atualização dinâmica do mapa e na prevenção de colisões.

Também se recomenda a realização de novos testes com o magnetômetro QMC5883L afastado fisicamente da Ponte H, dos cabos de alimentação dos motores e de outras possíveis fontes de interferência eletromagnética. Essa avaliação permitirá verificar se a instabilidade observada nas leituras de *heading* está associada principalmente ao posicionamento físico do sensor ou se decorre de limitações próprias do uso de magnetômetros em plataformas móveis com motores DC. Caso necessário, pode-se avaliar a substituição ou complementação do magnetômetro por uma unidade de medição inercial com giroscópio.

Em relação ao controle de movimento, trabalhos futuros podem aprofundar a calibração dos parâmetros PID e do fator de escala utilizado no controle de rotação. Para isso, recomenda-se a realização de ensaios repetidos de avanço, ré e curvas em diferentes ângulos, registrando o erro médio, o desvio padrão, o tempo de execução e a ocorrência de escorregamento das rodas. Esses dados permitiriam uma avaliação quantitativa mais rigorosa do desempenho do sistema e poderiam fundamentar ajustes mais precisos nos controladores.

A navegação também pode ser expandida por meio da implementação de atualização dinâmica do mapa de ocupação. Na versão atual, o mapa é utilizado principalmente como referência estática para planejamento de rotas e validação da navegação por células. Em versões futuras, as leituras do sensor de distância poderiam atualizar o estado das células em tempo real, permitindo que o robô replaneje trajetórias automaticamente quando obstáculos inesperados forem detectados.

Outra possibilidade de evolução consiste na melhoria da interface web, incluindo recursos de calibração diretamente pelo navegador, visualização gráfica dos logs, exportação dos dados dos testes e configuração de parâmetros como ganhos PID, escala de giro, tamanho da célula e limites de detecção de obstáculos. Esses recursos facilitariam a realização de ensaios experimentais e tornariam o sistema mais acessível para uso educacional e manutenção.

Por fim, recomenda-se ampliar a bateria de testes em ambientes físicos mais próximos de uma aplicação real, como corredores, salas e laboratórios. Esses testes devem considerar diferentes superfícies, níveis de iluminação, obstáculos estáticos e variações na posição inicial do robô. Com essas melhorias, o sistema poderá evoluir de um protótipo funcional validado em ambiente controlado para uma plataforma mais robusta de navegação autônoma em ambientes internos.

Referências

AUTOMATE.ORG. *Service Robots: Defense*. 2023. Disponível em: <<https://www.automate.org/robotics/service-robots/service-robots-defense>>.

CALISI, D. et al. Autonomous exploration for search and rescue robots. In: *Safety and Security Engineering II*. [S.l.]: WIT Press, 2007. p. 305–314.

CANTUCCI, F.; MARINI, M.; FALCONE, R. Effects of robot's adaptive autonomy on users experience in a museum scenario. In: *CEUR Workshop Proceedings*. [s.n.], 2024. v. 3735, p. 1–10. Disponível em: <https://ceur-ws.org/Vol-3735/paper_01.pdf>.

CARVALHO, F.; NETO, A. F. Sistemas embarcados: Fundamentos, aplicações e desafios tecnológicos. *Revista Matiz Online*, v. 14, p. 1–20, 2024. Instituto Matonense Municipal de Ensino Superior. Disponível em: <https://immes.edu.br/wp-content/uploads/2025/05/Artigo_MATIZ_2024_Sistemas_Embarcados.pdf>.

GONÇALVES, J.; SILVA, M. *Odometria visual para robôs*. [S.l.], 2017. Disponível em: <<https://www.ic.unicamp.br/~reltech/PFG/2017/PFG-17-19.pdf>>.

INDUSTRIES, A. *Adafruit VL53L0X Time of Flight Micro-LIDAR Distance Sensor Breakout*. [S.l.], 2024. Disponível em: <<https://cdn-learn.adafruit.com/downloads/pdf/adafruit-vl53l0x-micro-lidar-distance-sensor-breakout.pdf>>.

KUROSE, J. F.; ROSS, K. W. *Computer Networking: A Top-Down Approach*. 5. ed. USA: Addison-Wesley, 2009.

LEMOS, J. V. A. *Veículos Submersos Autônomos (AUV's): inspeção e manutenção de estruturas submarinas*. Tese (Doutorado) — Universidade Federal de Pernambuco, 2021. Disponível em: <<https://repositorio.ufpe.br/bitstream/123456789/48987/1/JO%C3%83O%20VICTOR%20ALMEIDA%20LEMOS.pdf>>.

MOUBARAK, P.; BEN-TZVI, P. Modular and reconfigurable mobile robotics. *Robotics and Autonomous Systems*, v. 60, n. 12, p. 1648–1663, 2012. ISSN 0921-8890. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0921889012001480>>.

NAUERT, S.; KAMPMANN, P. Inspection and maintenance of industrial infrastructure: challenges and perspectives for autonomous underwater intervention. *Frontiers in Robotics and AI*, v. 10, p. 1240276, 2023.

NEHMZOW, U. *Mobile robotics: a practical introduction*. [S.l.]: Springer Science & Business Media, 2012.

OLIVEIRA, A. *O uso do algoritmo Mapa de Rotas Probabilístico na robótica móvel*. Dissertação (Mestrado) — Universidade Federal de Lavras, 2020. Disponível em: <http://repositorio.ufla.br/bitstream/1/45521/1/MONOGRAFIA_O%20uso%20do%20algoritmo%20Mapa%20de%20Rotas%20Probabil%C3%ADstico%20na%20rob%C3%B3tica%20m%C3%B3vel.pdf>.

PARWEEN, R. et al. Autonomous self-reconfigurable floor cleaning robot. *IEEE Access*, v. 8, p. 114433–114442, 2020. ISSN 2169-3536.

PATEL, V. B. *Ultrasonic-SLAM*. 2021. <<https://github.com/PatelVatsalB21/Ultrasonic-SLAM>>. Acesso em: 27 jun. 2025.

SAMIR, A. et al. A review of features and characteristics of rescue robot with ai. *International Journal of Robotics*, v. 12, n. 4, p. 123–145, 2024.

SCHILLING, K. et al. Design of flexible autonomous transport robots for industrial production. In: *Proceedings of the IEEE International Symposium on Industrial Electronics (ISIE '97)*. [S.l.: s.n.], 1997. p. 791–796.

SIEGWART, R.; NOURBAKHSI, I. R.; SCARAMUZZA, D. *Introduction to Autonomous Mobile Robots*. 2. ed. Cambridge, MA: MIT press, 2011.

SILVA, L.; SOUZA, A. Robotics and ai in museums – the future of the present. *Extrica*, v. 12, n. 4, p. 45–56, 2024. Disponível em: <<https://www.extrica.com/article/24355>>.

STUDIES, T. H. C. for S. *The Military Applicability of Robotic and Autonomous Systems*. [S.l.], 2021. Disponível em: <https://hcss.nl/wp-content/uploads/2021/01/RAS_Military_Applicability_Final_.pdf>.

ÅSTRÖM, K. J.; HÄGGLUND, T. *Advanced PID Control*. Research Triangle Park, NC: ISA—The Instrumentation, Systems, and Automation Society, 2006.